

SQL for Software Engineers

A Scenario-Based SQL Guide for Freshers — ABC Technologies Case Study

Author: Skilling Team (KMIT)

Refcard: RC-SQL-2026

Published: 8 June 2026

Difficulty: Easy | **Estimated time:** 180 mins

Table of Contents

1. Section 1 — DATABASE FOUNDATIONS & EXPLORATION

- MODULE 1 - UNDERSTANDING THE DATABASE
- MODULE 2 - EXPLORING THE DATABASE
- CHAPTER REVIEW & SUMMARY

2. Section 2 — DATA RETRIEVAL AND FILTERING

- MODULE 3 - RETRIEVING DATA WITH SELECT
- MODULE 4 - FILTERING WITH WHERE
- CHAPTER REVIEW & SUMMARY

3. Section 3 — AGGREGATION AND GROUPING

- MODULE 5 - AGGREGATE FUNCTIONS
- MODULE 6 - GROUP BY AND HAVING
- CHAPTER REVIEW & SUMMARY

4. Section 4 — JOINS, SUBQUERIES AND WINDOW FUNCTIONS

- MODULE 7 - JOINS
- MODULE 8 - SUBQUERIES
- MODULE 9 - WINDOW FUNCTIONS
- CHAPTER REVIEW & SUMMARY

5. Section 5 — VIEWS, FUNCTIONS, STORED PROCEDURES AND TRANSACTIONS

- MODULE 10 - VIEWS
- MODULE 11 - FUNCTIONS
- MODULE 12 - STORED PROCEDURES
- MODULE 13 - TRANSACTIONS
- CHAPTER REVIEW & SUMMARY

6. Section 6 — INDEXES, METADATA AND PERFORMANCE

- MODULE 14 - INDEXES
- MODULE 15 - METADATA
- MODULE 16 - PERFORMANCE
- CHAPTER REVIEW & SUMMARY

Section 1 — DATABASE FOUNDATIONS & EXPLORATION

Welcome to ABC Technologies. You have just joined as a Junior Software Engineer, and your manager has handed you access to the `company_db` database. Before you write a single business query, you need to understand the ground you are standing on: what a database actually is, how it is organized into tables, rows, and columns, and how to navigate it confidently using MySQL's built-in inspection commands. This section walks you through that landscape end-to-end using the exact schema you will use for the rest of the Refcard: employees, departments, projects, `employee_projects`, customers, and orders.

MODULE 1 - UNDERSTANDING THE DATABASE

Your manager drops a message: 'You're on the `company_db` team starting Monday. Before you touch anything, make sure you understand what a database is, how our data is structured, and where the metadata lives.' This module answers those questions using the ABC Technologies schema as the running example.

EXPLANATION

> **Expert:** At ABC Technologies, every piece of business information — who the employees are, which department they belong to, which projects they work on, which customers placed orders — lives inside one MySQL database called `company_db`. A DBMS (Database Management System) like MySQL is the software that stores this data safely, lets multiple people query it at once, and enforces rules such as 'every employee must belong to a valid department'. As a Junior Software Engineer, you will not build the DBMS — you will use it to answer business questions in SQL.

CONCEPT CARD

> **Expert:** The vocabulary you will hear every day at ABC Technologies:

- **Database:** a named container. Ours is `company_db`.

- **Table:** a spreadsheet-like structure inside the database. `employees`, `departments`, `projects`, `employee_projects`, `customers`, `orders`.
- **Row (record):** one real-world thing. One row in `employees` = one person.
- **Column (field):** one attribute of that thing. `salary`, `city`, `email`.
- **Primary Key:** a column that uniquely identifies each row. `employee_id` in `employees`.
- **Foreign Key:** a column that points to a primary key in another table. `department_id` in `employees` points to `departments.department_id`.
- **Schema:** the blueprint — which tables exist, which columns they have, which keys connect them.
- **Metadata:** 'data about the data'. MySQL exposes it through the special database `INFORMATION_SCHEMA`.

ANALOGY

> **Expert:** Think of `company_db` as a filing cabinet. Each drawer is a table. Each folder in the drawer is a row. Each label on the folder is a column. The primary key is the unique folder number. The foreign key is a sticky note on one folder saying 'see folder #7 in the Departments drawer'. `INFORMATION_SCHEMA` is the master index card at the front of the cabinet that tells you what every drawer contains.

[Student] Student: So every business question I get from my manager basically becomes: 'find the right rows in the right tables using the right keys to connect them'?

[Expert] Expert: Exactly. That single sentence is the entire job. The rest of this Refcard is just learning the SQL syntax to do it fast.

What databases exist on this MySQL server?

Scenario. It is your first morning. Your manager tells you 'company_db is one of several databases on our MySQL server. Before you connect, list what's available so you know you are on the right box.'

Question. How do you list every database on the current MySQL server?

```
SHOW DATABASES;
```

Explanation. `SHOW DATABASES` returns one row per database the logged-in user can see. You will use it to confirm `company_db` is present before you start working.

Which database am I currently connected to?

Scenario. You opened two terminal tabs yesterday. Before you run a DELETE, you want to be 100% sure the current tab is pointed at `company_db` and not a production clone.

Question. How do you check which database is currently selected?

```
SELECT DATABASE();
```

Explanation. `DATABASE()` is a built-in function that returns the name of the current database or NULL if none is selected. Always run this before any destructive query.

What tables exist inside company_db?

Scenario. Your manager says 'we recently added a projects module. Confirm the tables in company_db so you know what is available to query.'

Question. How do you list every table inside company_db?

```
USE company_db;  
SHOW TABLES;
```

Explanation. `USE` sets the current database. `SHOW TABLES` then lists every table in it. You should see employees, departments, projects, employee_projects, customers, orders.

How many tables are in company_db?

Scenario. A quick sanity check before your first standup: 'How many tables are we dealing with in company_db?'

Question. How do you get the exact count of tables in company_db?

```
SELECT COUNT(*) AS total_tables  
FROM information_schema.tables  
WHERE table_schema = 'company_db';
```

Explanation. `information_schema.tables` has one row per table in every database. Filtering by `table_schema = 'company_db'` and counting gives you the exact number.

What columns does the employees table have?

Scenario. Before writing your first query on employees, you want to see every column, its data type, and whether it can be NULL.

Question. How do you inspect the structure of the employees table?

```
DESCRIBE employees;
```

Explanation. `DESCRIBE` (or `DESC`) lists every column with its data type, NULL setting, key info, default value, and extra flags — the fastest way to learn a table's shape.

How was the orders table originally created?

Scenario. A senior developer wants the exact `CREATE TABLE` statement for `orders` so she can replicate it in a staging environment.

Question. How do you retrieve the original `CREATE TABLE` statement for `orders`?

```
SHOW CREATE TABLE orders;
```

Explanation. `SHOW CREATE TABLE` returns the full DDL statement including columns, keys, constraints, and the storage engine — perfect for cloning a table's definition.

Which table has the most rows?

Scenario. Your manager wants a rough sense of scale: which table in `company_db` has the most records? It helps decide where indexing effort should go first.

Question. How do you find the table with the highest row count in `company_db`?

```
SELECT table_name, table_rows
FROM information_schema.tables
WHERE table_schema = 'company_db'
ORDER BY table_rows DESC
LIMIT 1;
```

Explanation. `information_schema.tables.table_rows` gives an approximate row count per table. Sorting descending and taking the top row surfaces the largest table.

WARNING

> **Expert:** `table_rows` in `INFORMATION_SCHEMA` is an **estimate** for InnoDB tables — accurate enough for capacity planning but not for reporting. For an exact count of one table, always use `SELECT COUNT(*) FROM <table>;` .

COMMON MISTAKE

> **Expert:** Freshers often forget to run `USE company_db;` first, then wonder why `SHOW TABLES` returns nothing or shows tables from a different database. Rule of thumb: the first line of every new session is `USE company_db;` .

QUIZ

> **Expert: Q:** You want to list every table in `company_db` without switching databases. Which query works?

A) `SHOW TABLES;` B) `SHOW TABLES FROM company_db;` C) `SELECT * FROM tables;`

Answer: B. `SHOW TABLES FROM company_db;` lets you inspect any database without changing your current session.

EXERCISE

> **Expert:** 1. List all databases on your MySQL server.

2. Switch to `company_db` and confirm with `SELECT DATABASE();`

3. List all tables.

4. Describe every table one by one and write down which column is the primary key.

Figure 1.1 — The `company_db` schema at a glance — A clean entity-relationship diagram showing six rectangles arranged on a light background. Top-left: `departments` (`department_id` PK, `department_name`). Center: `employees` (`employee_id` PK, `first_name`, `last_name`, `gender`, `age`, `email`, `phone`, `salary`, `hire_date`, `department_id` FK, `manager_id` FK-self, `city`). Top-right: `projects` (`project_id` PK, `project_name`, `budget`, `department_id` FK). Center-right: `employee_projects` (`employee_id` FK, `project_id` FK) shown as a junction table connecting employees and projects. Bottom-left: `customers` (`customer_id` PK, `customer_name`, `city`). Bottom-right: `orders` (`order_id` PK, `customer_id` FK, `order_date`, `amount`). Solid arrows connect: `employees.department_id` -> `departments`, `projects.department_id` -> `departments`, `employee_projects` to both `employees` and `projects`, `orders.customer_id` -> `customers`, and a curved self-loop on `employees.manager_id` -> `employees.employee_id`. Primary keys are marked with a small gold key icon; foreign keys with a silver key icon.

MODULE 2 - EXPLORING THE DATABASE

Now that you understand the vocabulary, this module gives you the exact commands you will type a hundred times a week at ABC Technologies to explore any database quickly — the commands a senior developer runs on autopilot.

EXPLANATION

> **Expert:** Exploration is the first skill a Junior Software Engineer needs. Before you fix a bug, before you write a report, before you optimize a slow query — you must know what tables exist, how they are shaped, how big they are, and how they connect. MySQL gives you two families of commands for this: quick `SHOW` and `DESCRIBE` statements for day-to-day work, and the `INFORMATION_SCHEMA` views for programmatic answers.

Show me the list of databases

Scenario. You just SSH'd into a new MySQL server your manager provisioned. Confirm which databases live on it.

Question. How do you list every database?

```
SHOW DATABASES;
```

Explanation. Returns one row per database. On ABC Technologies servers you should see at minimum `company_db`, plus `information_schema`, `mysql`, `performance_schema`, and `sys` which are system databases.

Select the company_db database

Scenario. You want every subsequent query to run against `company_db` without prefixing table names.

Question. How do you set `company_db` as the current database?

```
USE company_db;
```

Explanation. `USE` changes the default database for the current session. After this, `SELECT * FROM employees` works without writing `company_db.employees`.

Confirm the currently selected database

Scenario. You are about to run a bulk update. Double-check you are in the right database.

Question. How do you display the current database name?

```
SELECT DATABASE();
```

Explanation. Returns the active database name or `NULL`. Make this the first line of any sensitive script — a 5-second habit that prevents disasters.

List all tables

Scenario. You need to remember all the table names before starting today's ticket.

Question. How do you list every table in the current database?

```
SHOW TABLES;
```

Explanation. Prints one row per table in the current database. Combine with `USE company_db` to see the six ABC Technologies business tables.

Describe the projects table

Scenario. Before writing a join between projects and departments, you want to see projects' columns and types.

Question. How do you view the structure of the projects table?

```
DESCRIBE projects;
```

Explanation. Shows column name, data type, nullability, key info, default, and extras. `DESC projects;` is the short form and works identically.

Show columns of the customers table with more detail

Scenario. You want the same information as DESCRIBE, but you find `SHOW COLUMNS` easier to filter and script against.

Question. How do you list every column in customers?

```
SHOW COLUMNS FROM customers;
```

Explanation. Equivalent to DESCRIBE but supports a `LIKE` clause, e.g. `SHOW COLUMNS FROM customers LIKE 'city';` for pattern-based filtering.

See the CREATE TABLE definition of employees

Scenario. A colleague asks for the exact DDL of `employees` so she can create the same table in her local sandbox.

Question. How do you retrieve the CREATE statement of employees?

```
SHOW CREATE TABLE employees;
```

Explanation. Returns two columns: table name and the full CREATE TABLE SQL — columns, keys, constraints, engine, and character set. Copy-paste ready.

Check the size of every table on disk

Scenario. The DBA is planning storage for next quarter and asks: 'How much disk does each table in company_db take?'

Question. How do you list every table with its data and index size in MB?

```
SELECT table_name,  
       ROUND((data_length + index_length) / 1024 / 1024, 2) AS size_mb  
FROM information_schema.tables  
WHERE table_schema = 'company_db'  
ORDER BY size_mb DESC;
```

Explanation. `data_length` is the row storage and `index_length` is the index storage, both in bytes. Dividing by 1024*1024 converts to MB and rounding to 2 decimals keeps the report clean.

Count rows in every table

Scenario. Your manager wants a one-line summary for each table: how many rows do we have?

Question. How do you list every table with an approximate row count?

```
SELECT table_name, table_rows  
FROM information_schema.tables  
WHERE table_schema = 'company_db'  
ORDER BY table_rows DESC;
```

Explanation. `table_rows` is a fast estimate for InnoDB tables. Perfect for a scale overview; for an exact count of one table run `SELECT COUNT(*) FROM <table>;`.

See full table metadata

Scenario. The DBA wants engine, collation, auto-increment value, and creation time for every table in company_db.

Question. How do you list rich metadata for every table?

```
SHOW TABLE STATUS FROM company_db;
```

Explanation. One row per table with engine, row count, average row length, data/index size, auto-increment value, and timestamps. The single fastest health check on a schema.

MEMORY TRICK CARD

> **Expert:** Remember exploration as the ladder **D-T-C-S**: **D**atabases (`SHOW DATABASES`), **T**ables (`SHOW TABLES`), **C**olumns (`DESCRIBE <t>`), **S**tatus (`SHOW TABLE STATUS`). Every day you climb the same ladder from top to bottom.

EXAM TIP CARD

> **Expert:** Interview favorite: 'How would you find the largest table in a database?' The expected answer uses `information_schema.tables` with `data_length + index_length` ordered DESC — exactly the query above.

AI PROMPT

> **Expert:** Try this prompt in Claude or ChatGPT: 'Given a MySQL database called `company_db` with tables `employees`, `departments`, `projects`, `employee_projects`, `customers`, and `orders`, generate a single query using `information_schema` that lists every foreign key and the table it references.' Then verify the output against `SHOW CREATE TABLE` .

Figure 1.2 — Exploration command cheat-sheet — A two-column reference card on a light background. Left column header 'Quick SHOW commands' lists: `SHOW DATABASES`, `USE company_db`, `SELECT DATABASE()`, `SHOW TABLES`, `DESCRIBE employees`, `SHOW COLUMNS FROM employees`, `SHOW CREATE TABLE employees`, `SHOW INDEX FROM employees`, `SHOW TABLE STATUS`. Right column header 'INFORMATION_SCHEMA queries' lists: `SELECT * FROM information_schema.schemata`, `information_schema.tables`, `information_schema.columns`, `information_schema.key_column_usage`, `information_schema.table_constraints`. Between the two columns a vertical divider with the caption 'Quick shell answers <--> Programmatic answers'. A small footer note reads 'Use SHOW for daily exploration; use INFORMATION_SCHEMA when the answer feeds another query or a report.'

CHAPTER REVIEW & SUMMARY

EXPLANATION

> **Expert:** In Section 1 you learned the geography of ABC Technologies' data: what a DBMS is, what tables/rows/columns/keys mean, and how the six business tables of `company_db` connect. You then picked up the daily exploration commands — `SHOW DATABASES`, `USE`, `SELECT DATABASE()`, `SHOW TABLES`, `DESCRIBE`, `SHOW CREATE TABLE`, and `INFORMATION_SCHEMA` queries for programmatic answers about size and row counts. This is the toolkit every Junior Software Engineer at ABC Technologies opens the session with. From Section 2 onwards you will start pulling actual business data out of these tables using `SELECT`.

CONCEPT CARD

> **Expert:** Key takeaways:

- The one database you will use throughout this Refcard is `company_db`.
- Six business tables: `employees`, `departments`, `projects`, `employee_projects`, `customers`, `orders`.
- Primary keys uniquely identify rows; foreign keys wire tables together.
- `SHOW` commands for quick shell exploration; `INFORMATION_SCHEMA` when the answer feeds a script or report.
- Always run `SELECT DATABASE();` before any destructive statement.

Section 2 — DATA RETRIEVAL AND FILTERING

Now that you can navigate `company_db`, your manager starts assigning real tickets: 'Give me a list of Hyderabad employees', 'Show me everyone earning above 80,000', 'Find employees whose name starts with A'. Every one of these questions is a `SELECT` with a `WHERE` clause. Section 2 walks you through `SELECT` (retrieval) and `WHERE` (filtering) using the exact tickets a junior at ABC Technologies gets in the first three weeks.

MODULE 3 - RETRIEVING DATA WITH SELECT

`SELECT` is the workhorse of SQL. Every report, every dashboard, every API endpoint at ABC Technologies is ultimately powered by a `SELECT` statement. This module covers `SELECT`, `DISTINCT`, `LIMIT`, aliases, and `ORDER BY` through fifteen real tickets.

EXPLANATION

> **Expert:** The pattern is always the same: SELECT tells MySQL *what columns* you want, FROM tells it *which table*, and optional clauses like DISTINCT, ORDER BY, and LIMIT reshape the result. Learn this five-word skeleton and 80% of daily work becomes muscle memory:

```
SELECT ... FROM ... WHERE ... ORDER BY ... LIMIT ... .
```

CONCEPT CARD

> **Expert:** The five clauses you will use most from Module 3:

- **SELECT col1, col2** — pick specific columns.
- ****SELECT ***** — pick every column (avoid in production; details in Module 16).
- **DISTINCT** — remove duplicate rows from the result.
- **AS alias** — rename a column in the output.
- **ORDER BY col ASC|DESC** — sort the result.
- **LIMIT n** — cap the number of rows returned.
- **LIMIT offset, n** — skip `offset` rows then return `n` (pagination).

Show all employees

Scenario. First ticket ever: 'Pull the full employees list so HR can print an ID card audit.'

Question. How do you retrieve every column of every row in employees?

```
SELECT * FROM employees;
```

Explanation. `SELECT *` returns every column of the table. It is fine for exploration but you will replace `*` with explicit column names for any query that goes into production code.

Show only employee names

Scenario. HR only wants first and last names for a phone-list mail-merge.

Question. How do you fetch just `first_name` and `last_name` from employees?

```
SELECT first_name, last_name FROM employees;
```

Explanation. Listing columns explicitly returns exactly what is needed — faster to transfer, safer against schema changes, easier to read.

Show unique cities where employees live

Scenario. The travel desk wants a distinct list of employee cities to plan regional meetups.

Question. How do you list every distinct city in employees?

```
SELECT DISTINCT city FROM employees;
```

Explanation. `DISTINCT` removes duplicate rows from the result. Perfect when the same city appears many times but you only need each once.

Show first five employees

Scenario. For a quick smoke test of a new dashboard, you need just five sample rows.

Question. How do you retrieve the first five employees only?

```
SELECT * FROM employees LIMIT 5;
```

Explanation. `LIMIT 5` caps the result at five rows. Without an `ORDER BY`, MySQL is free to pick any five; add `ORDER BY` when the order matters.

Show the latest hired employee

Scenario. Your manager wants the name of the most recently hired employee for a welcome email.

Question. How do you find the employee hired most recently?

```
SELECT first_name, last_name, hire_date  
FROM employees  
ORDER BY hire_date DESC  
LIMIT 1;
```

Explanation. Sort by `hire_date` descending, then `LIMIT 1` to pick the top row — the newest hire.

Show the highest salary in the company

Scenario. Finance wants the maximum single salary being paid so they can benchmark against industry median.

Question. How do you show the highest salary?

```
SELECT salary
FROM employees
ORDER BY salary DESC
LIMIT 1;
```

Explanation. Order by salary descending and take the top row. In Module 5 you will learn `MAX(salary)` which is more idiomatic for this need.

Show the lowest salary in the company

Scenario. Same finance ticket — they also want the minimum salary paid across the company.

Question. How do you find the smallest salary value?

```
SELECT salary
FROM employees
ORDER BY salary ASC
LIMIT 1;
```

Explanation. ORDER BY ascending puts the smallest value first; LIMIT 1 keeps only that row.

Rename columns using aliases

Scenario. A Google Sheets integration expects headers `Name` and `Monthly Pay` — not `first_name` and `salary`.

Question. How do you rename columns in the query output?

```
SELECT first_name AS Name,
       salary AS `Monthly Pay`
FROM employees;
```

Explanation. `AS` renames a column in the result set only — the table itself is untouched. Wrap aliases with spaces in backticks.

Show the second page of employees

Scenario. The employee directory UI shows 10 per page. The user just clicked page 2.

Question. How do you return rows 11 to 20 of employees ordered by ID?

```
SELECT *  
FROM employees  
ORDER BY employee_id  
LIMIT 10 OFFSET 10;
```

Explanation. `LIMIT 10 OFFSET 10` skips the first 10 rows then returns the next 10 — standard server-side pagination pattern.

Sort employees by name

Scenario. The HR employee-directory page must display everyone alphabetically by last name, then first name.

Question. How do you sort employees by two columns?

```
SELECT first_name, last_name  
FROM employees  
ORDER BY last_name ASC, first_name ASC;
```

Explanation. `ORDER BY` accepts multiple columns. MySQL sorts by `last_name` first, then `first_name` breaks ties — exactly like alphabetizing a phone book.

WARNING

> **Expert:** Avoid `SELECT *` in production for two reasons: (1) it transfers data you may not need, wasting bandwidth; (2) if someone adds a column to the table, your app silently receives extra data it never asked for. Always list the columns you want.

COMMON MISTAKE

> **Expert:** `SELECT DISTINCT city, first_name FROM employees;` does not give you distinct cities. `DISTINCT` applies to the **combination of all selected columns**, so any two rows that differ in either column are kept. For unique cities alone, select only `city`.

MEMORY TRICK CARD

> **Expert:** Remember `SELECT` clause order with the phrase **Smart Foxes Walk Gently Hoping Others Listen**: `SELECT`, `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`.

EXERCISE

> **Expert:** 1. List employee_id, first_name, and city for every employee, sorted by city.

2. Show unique departments referenced in employees using DISTINCT.
3. Return the 3 highest-paid employees using ORDER BY + LIMIT.
4. Paginate: return rows 21-30 of employees ordered by employee_id.

Figure 2.1 — Anatomy of a SELECT statement — A horizontal syntax diagram on a soft off-white background. Six labeled boxes arranged left to right with arrows: (1) SELECT [DISTINCT] columns, (2) FROM table, (3) WHERE conditions, (4) GROUP BY, (5) ORDER BY, (6) LIMIT. Above each box a small caption explains its role: 'what to show', 'from where', 'which rows', 'bucket rows', 'sort order', 'cap rows'. Below the diagram a dashed line labeled 'Logical execution order' with numbers showing that MySQL evaluates FROM -> WHERE -> GROUP BY -> SELECT -> ORDER BY -> LIMIT, which differs from the writing order. The SELECT and FROM boxes are highlighted in a warm gold to show they are mandatory; the rest are shown in muted gray to indicate optional.

MODULE 4 - FILTERING WITH WHERE

'Show me employees from Hyderabad.' 'Only employees earning above 80,000.' 'Only Gmail users.' Every one of these tickets narrows the rows using a WHERE clause. This module covers WHERE, IN, BETWEEN, LIKE, NULL, and the logical operators AND/OR/NOT across fifteen realistic tickets.

EXPLANATION

> **Expert:** WHERE runs a boolean test on each row and keeps the ones that pass. If the test returns TRUE, the row is included. FALSE or NULL — excluded. Understand this and the strange behavior of NULL later in the module stops being strange.

CONCEPT CARD

> **Expert:** The operators you will use in WHERE every day:

- =, <>, <, <=, >, >= — comparison.
- AND, OR, NOT — combine conditions.
- IN (a, b, c) — value belongs to a list.
- BETWEEN x AND y — value in an inclusive range.
- LIKE 'A%' — string pattern (% = any chars, _ = one char).

- **IS NULL / IS NOT NULL** — check for missing values.

Precedence rule: NOT > AND > OR. Use parentheses freely — clarity beats cleverness.

Employees from Hyderabad

Scenario. The Hyderabad office is planning a Friday team lunch. HR needs the list of employees based there.

Question. How do you find all employees in Hyderabad?

```
SELECT * FROM employees WHERE city = 'Hyderabad';
```

Explanation. The WHERE clause keeps only rows where city equals 'Hyderabad'. String comparison in MySQL is case-insensitive by default with standard collations.

Employees earning more than 50000

Scenario. Finance is auditing compensation and wants everyone earning above 50,000.

Question. How do you list employees with salary greater than 50000?

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary > 50000;
```

Explanation. Comparison operators work on numeric columns. Strictly greater than uses `>`; for greater-than-or-equal use `>=`.

Employees earning between 40000 and 80000

Scenario. Finance now wants the mid-band: employees between 40k and 80k salary.

Question. How do you filter salary within a range?

```
SELECT first_name, salary
FROM employees
WHERE salary BETWEEN 40000 AND 80000;
```

Explanation. `BETWEEN x AND y` is **inclusive** on both ends — same as `salary >= 40000 AND salary <= 80000` but shorter and self-documenting.

Employees in HR or IT departments

Scenario. Assume department_id 1 is HR and 2 is IT. Your manager wants everyone from those two departments.

Question. How do you filter rows where a column equals one of several values?

```
SELECT first_name, department_id
FROM employees
WHERE department_id IN (1, 2);
```

Explanation. `IN (list)` is cleaner than `department_id = 1 OR department_id = 2` and scales well when the list grows to a dozen values.

Employees NOT in HR

Scenario. For a technical training rollout, HR themselves should be excluded from the invitation list.

Question. How do you exclude employees from a specific department?

```
SELECT first_name, department_id
FROM employees
WHERE department_id NOT IN (1);
```

Explanation. `NOT IN` inverts the membership test. Also works with a single value; here 1 is HR so we exclude just HR.

Employees whose first name starts with A

Scenario. The company newsletter wants to feature everyone whose name starts with 'A' this month.

Question. How do you filter first names that begin with 'A'?

```
SELECT first_name
FROM employees
WHERE first_name LIKE 'A%';
```

Explanation. `LIKE` with `%` at the end matches anything that starts with 'A'. `%` = zero or more characters, `_` = exactly one character.

Employees whose last name ends with 'Kumar'

Scenario. For a South-India cultural event, pull employees whose last name ends with 'Kumar'.

Question. How do you match strings ending with a specific text?

```
SELECT first_name, last_name
FROM employees
WHERE last_name LIKE '%Kumar';
```

Explanation. % at the beginning of the pattern allows any prefix; the match must end with 'Kumar'. Case is handled per collation — usually insensitive.

Employees whose name contains 'Raj'

Scenario. You need to find every employee whose first name has 'Raj' anywhere in it — Rajesh, Rajat, Neeraj, etc.

Question. How do you match a substring inside a string?

```
SELECT first_name
FROM employees
WHERE first_name LIKE '%Raj%';
```

Explanation. % at both ends means 'anything before Raj, then Raj, then anything after' — the classic substring search.

Employees whose email is missing

Scenario. IT is running a cleanup and wants employees who never had an email set up.

Question. How do you find rows where a column is NULL?

```
SELECT first_name, last_name
FROM employees
WHERE email IS NULL;
```

Explanation. NULL means 'unknown'. You must use `IS NULL`, never `= NULL` — the latter always returns unknown, so no rows match.

Employees whose phone is set

Scenario. Ops wants to build an SMS distribution list — only employees who actually have a phone number.

Question. How do you find rows where a column has any non-NULL value?

```
SELECT first_name, phone
FROM employees
WHERE phone IS NOT NULL;
```

Explanation. `IS NOT NULL` keeps rows where a value exists. Combine with `LIKE` if you also want to validate the format.

Female employees from Hyderabad

Scenario. The women-in-tech circle plans a Hyderabad meetup. Get the guest list.

Question. How do you combine two conditions with `AND`?

```
SELECT first_name, gender, city
FROM employees
WHERE gender = 'F' AND city = 'Hyderabad';
```

Explanation. `AND` requires **both** sides to be true. If either is false or `NULL`, the row is excluded.

IT employees earning above 80000

Scenario. IT leadership wants a shortlist of high earners in their own department (IT = department_id 2).

Question. How do you filter on department `AND` salary at the same time?

```
SELECT first_name, department_id, salary
FROM employees
WHERE department_id = 2 AND salary > 80000;
```

Explanation. `AND` chains any number of conditions. Order does not matter for correctness, but putting the most selective condition first often helps performance.

Employees hired after 2023-01-01

Scenario. Onboarding wants a list of everyone who joined in 2023 or later.

Question. How do you filter by a date threshold?

```
SELECT first_name, hire_date
FROM employees
WHERE hire_date >= '2023-01-01';
```

Explanation. Use ISO format `YYYY-MM-DD` for date literals. `>=` includes anyone hired on January 1st, 2023 as well.

Employees hired this year

Scenario. For the annual all-hands, mark employees hired in the current calendar year.

Question. How do you filter rows hired in the current year?

```
SELECT first_name, hire_date
FROM employees
WHERE YEAR(hire_date) = YEAR(CURDATE());
```

Explanation. `YEAR(hire_date)` extracts the year; `CURDATE()` returns today's date. Comparing the two matches only current-year hires.

Gmail users

Scenario. For a Google Workspace license audit, get every employee whose email is on gmail.com.

Question. How do you find employees whose email ends with '@gmail.com'?

```
SELECT first_name, email
FROM employees
WHERE email LIKE '%@gmail.com';
```

Explanation. `LIKE` with `%@gmail.com` catches every string ending with that domain regardless of the local part.

Employees from Hyderabad or Bangalore

Scenario. A South-India engineering sync needs everyone based in either Hyderabad or Bangalore.

Question. How do you match rows where a column is one of several values?

```
SELECT first_name, city
FROM employees
WHERE city IN ('Hyderabad', 'Bangalore');
```

Explanation. `IN` is cleaner than `city = 'Hyderabad' OR city = 'Bangalore'` and scales easily as the list grows.

WARNING

> **Expert:** Never write `WHERE column = NULL`. It looks correct but always evaluates to unknown so it returns zero rows — even for actual NULL values. Use `IS NULL` and `IS NOT NULL` every single time.

COMMON MISTAKE

> **Expert:** `WHERE city = 'Hyderabad' OR city = 'Bangalore' AND salary > 80000` does NOT mean 'from either city with salary > 80k'. Because AND binds tighter than OR, MySQL reads it as `city='Hyderabad' OR (city='Bangalore' AND salary>80000)`. Fix with parentheses: `(city='Hyderabad' OR city='Bangalore') AND salary>80000`.

QUIZ

> **Expert: Q:** Which query correctly returns employees whose phone number is not recorded?

A) `SELECT * FROM employees WHERE phone = NULL;` B) `SELECT * FROM employees WHERE phone != NULL;` C) `SELECT * FROM employees WHERE phone IS NULL;`

Answer: C. Only `IS NULL` correctly tests for missing values.

EXAM TIP CARD

> **Expert:** Interviewers love the `LIKE 'A%'` vs `LIKE '%A'` vs `LIKE '%A%'` distinction. Memorize: `%` at the end = starts with, at the start = ends with, at both ends = contains.

EXERCISE

> **Expert:** 1. Find all employees with salary between 60000 and 100000 in Hyderabad or Bangalore.

2. Find employees whose email is not on gmail.com and whose phone is NOT NULL.
3. Find female employees hired before 2020 whose first name starts with 'S'.
4. Find employees where department_id is either 1, 3, or 5 AND salary is above average (rough guess: > 70000).

AI PROMPT

> **Expert:** Try this in Claude: 'For an employees table with columns (employee_id, first_name, last_name, gender, salary, city, hire_date, department_id), write a WHERE clause that returns female employees from Hyderabad or Bangalore, hired in the last three years, earning between 50k and 120k, whose email is missing.' Then verify the output uses IS NULL and parenthesized OR correctly.

Figure 2.2 — WHERE operators cheat sheet — A 3-column table on a light background. Left column 'Operator' lists =, <>, IN, BETWEEN, LIKE, IS NULL, AND, OR, NOT. Middle column 'ABC Technologies example' shows one ready-to-use snippet per operator: city = 'Hyderabad'; salary <> 0; department_id IN (1, 2, 3); salary BETWEEN 40000 AND 80000; first_name LIKE 'A%'; email IS NULL; city = 'Hyderabad' AND gender = 'F'; city = 'Hyderabad' OR city = 'Bangalore'; NOT (department_id = 5). Right column 'Reads as' explains each in plain English: 'exactly Hyderabad', 'not zero', 'in HR, IT, or Finance', 'salary between 40k and 80k inclusive', 'name starts with A', 'email is missing', 'women from Hyderabad', 'employees in Hyderabad or Bangalore', 'anyone not in dept 5'. A footer note in small text reads 'Precedence: NOT > AND > OR — use parentheses when unsure.'

CHAPTER REVIEW & SUMMARY

EXPLANATION

> **Expert:** Section 2 gave you the two clauses you will touch in every SQL script for the rest of your career: SELECT to choose columns and WHERE to choose rows. Alongside them you learned DISTINCT (deduplicate), ORDER BY (sort), LIMIT/OFFSET (cap and paginate), and aliases (rename output). You also picked up the operators that make WHERE powerful: comparison, IN, BETWEEN, LIKE, IS NULL, and AND/OR/NOT with their precedence rules.

CONCEPT CARD

> **Expert:** Key takeaways:

- SELECT explicit columns, not *, for production queries.
- DISTINCT applies to the whole row of the result — not a single column.
- ORDER BY ... LIMIT n is the pattern for top-N.
- LIMIT n OFFSET k is standard pagination.
- NULL is 'unknown' — use IS NULL, never = NULL.
- AND binds tighter than OR — parenthesize mixed logic.

- LIKE 'A%' starts-with, '%A' ends-with, '%A%' contains.

From Section 3 onward you will start aggregating: how many employees per department, what is the total order value per city, and so on.

Section 3 — AGGREGATION AND GROUPING

Your manager stops asking for lists and starts asking for numbers. 'How many employees do we have?' 'What's the average salary?' 'What's the total order value from Hyderabad customers?' 'Which departments have more than 10 employees?' Each of these questions collapses many rows into one summary row. This is aggregation, powered by COUNT, SUM, AVG, MIN, MAX, and shaped by GROUP BY and HAVING. Section 3 covers everything you need to build reports at ABC Technologies.

MODULE 5 - AGGREGATE FUNCTIONS

Aggregate functions take many rows and return one number. COUNT the rows, SUM a column, AVG a column, or find MIN/MAX. This module walks you through every aggregate through real ABC Technologies reports.

EXPLANATION

> **Expert:** An aggregate function reads a whole set of rows and returns a single value.

SELECT COUNT(*) FROM employees; reads every row and returns one number. When you add GROUP BY (next module), the aggregate runs once per group instead of once across the whole table — which is how you get 'employees per department' reports.

CONCEPT CARD

> **Expert:** The five aggregates you will use every day at ABC Technologies:

- **COUNT(*)** — total rows.
- **COUNT(col)** — rows where col is NOT NULL.
- **COUNT(DISTINCT col)** — number of unique non-NULL values.
- **SUM(col)** — total.
- **AVG(col)** — average.
- **MIN(col), MAX(col)** — smallest and largest.

All of them **ignore NULLs** except `COUNT(*)`, which counts every row regardless.

Total number of employees

Scenario. Ops needs the current headcount for a quarterly report.

Question. How do you count all employees?

```
SELECT COUNT(*) AS total_employees FROM employees;
```

Explanation. `COUNT(*)` counts every row of the table, NULLs included. The alias `total_employees` makes the report column self-explanatory.

Number of employees with email on file

Scenario. IT wants to know how many employees actually have an email set — some are blank.

Question. How do you count only rows where email is not NULL?

```
SELECT COUNT(email) AS employees_with_email FROM employees;
```

Explanation. `COUNT(column)` counts rows where the column is NOT NULL. The gap between this and `COUNT(*)` tells you how many employees are missing an email.

Total departments

Scenario. A quick org-chart check: how many departments does ABC Technologies have?

Question. How do you count the departments?

```
SELECT COUNT(*) AS total_departments FROM departments;
```

Explanation. Same pattern as employees. `COUNT(*)` returns the row count of the departments table.

Total salary paid every month

Scenario. Finance is preparing the monthly payroll summary and wants the total salary outflow.

Question. How do you sum every employee's salary?

```
SELECT SUM(salary) AS total_payroll FROM employees;
```

Explanation. `SUM` adds every non-NULL value. If any salary is NULL it is silently ignored — usually the desired behavior for money columns.

Average salary

Scenario. HR is benchmarking against industry data and wants the company's average salary.

Question. How do you compute the average salary?

```
SELECT AVG(salary) AS average_salary FROM employees;
```

Explanation. `AVG` = `SUM` / `COUNT` restricted to non-NULL values. Wrap with `ROUND(AVG(salary), 2)` when the report needs two decimal places.

Highest salary using MAX

Scenario. Same finance ticket as Module 3, but written the idiomatic way with MAX.

Question. How do you find the highest salary using an aggregate?

```
SELECT MAX(salary) AS highest_salary FROM employees;
```

Explanation. `MAX` returns the largest non-NULL value. Compared to `ORDER BY ... LIMIT 1`, it is a single scan and clearer about intent.

Lowest salary using MIN

Scenario. The mirror image: the minimum salary in the company.

Question. How do you find the smallest salary using an aggregate?

```
SELECT MIN(salary) AS lowest_salary FROM employees;
```

Explanation. `MIN` returns the smallest non-NULL value. Ignores NULLs entirely — perfect for salary.

Highest single order value

Scenario. The sales team wants to celebrate the biggest single order ever placed.

Question. How do you find the largest amount in the orders table?

```
SELECT MAX(amount) AS biggest_order FROM orders;
```

Explanation. Same pattern applied to a different table. `MAX(amount)` returns the largest order value in one row.

Average order amount

Scenario. Sales leadership wants the average deal size for capacity planning.

Question. How do you compute the average order amount?

```
SELECT ROUND(AVG(amount), 2) AS average_order FROM orders;
```

Explanation. Rounding to two decimals makes the result presentation-ready without a second step in the reporting layer.

Number of distinct cities employees live in

Scenario. The travel desk wants a single number: how many different cities do our employees live in?

Question. How do you count unique cities in employees?

```
SELECT COUNT(DISTINCT city) AS distinct_cities  
FROM employees;
```

Explanation. `COUNT(DISTINCT col)` counts unique non-NULL values. Much more direct than a sub-query on DISTINCT.

Number of customers

Scenario. The sales dashboard needs today's customer count.

Question. How do you count all customers?

```
SELECT COUNT(*) AS total_customers FROM customers;
```

Explanation. `COUNT(*)` returns the total row count of customers, which equals the number of customers because `customer_id` is unique.

Total revenue from all orders

Scenario. The CEO opens Monday's dashboard: what is the lifetime revenue from orders?

Question. How do you sum the amount column across every order?

```
SELECT SUM(amount) AS total_revenue FROM orders;
```

Explanation. SUM ignores NULLs and returns a single total. Add a WHERE clause on `order_date` for period-scoped revenue.

Multiple aggregates in one row

Scenario. You want a single summary row: total, average, min, and max salary in one shot.

Question. How do you compute several aggregates in one query?

```
SELECT COUNT(*) AS employees,  
       SUM(salary) AS payroll,  
       ROUND(AVG(salary), 2) AS avg_salary,  
       MIN(salary) AS min_salary,  
       MAX(salary) AS max_salary  
FROM employees;
```

Explanation. SELECT can list any number of aggregates in one row. Perfect for a KPI card at the top of a dashboard.

WARNING

> **Expert:** `AVG(salary)` ignores NULL salaries — that changes the denominator. If ten employees have NULL salary out of 100, the average is over 90 rows, not 100. If you want to treat NULL as 0, use `AVG(COALESCE(salary, 0))`.

COMMON MISTAKE

> **Expert:** `SELECT first_name, MAX(salary) FROM employees;` looks like it should return the name of the highest-paid person. It does NOT — without `GROUP BY` it returns one row with `MAX(salary)` correct but `first_name` unrelated (any value MySQL picks). To get the top-earner's name, use `ORDER BY salary DESC LIMIT 1` or a subquery — you will see subqueries in Section 4.

MEMORY TRICK CARD

> **Expert:** The five aggregates spell **CSAMM**: Count, Sum, Average, Min, Max. Or remember the phrase 'Careful SQL Aggregates Miss NULLs' — every aggregate except `COUNT(*)` skips NULLs.

Figure 3.1 — How an aggregate collapses rows — A two-panel illustration on a light background. Left panel labeled 'Input rows' shows a small vertical stack of six employee rectangles with salary values 40000, 50000, 60000, 70000, 80000, 90000, each with an employee icon. A large downward arrow in the middle is labeled `AVG(salary)`. Right panel

labeled 'Aggregate output' shows a single result rectangle with the value 65000 highlighted in a gold badge. Below the arrow a small caption: 'One value per group. Without GROUP BY, the whole table is one group.' The caption underneath compares COUNT(*), SUM(salary), MIN(salary), MAX(salary) side by side to show they all collapse to one number: 6, 390000, 40000, 90000.

MODULE 6 - GROUP BY AND HAVING

Aggregates over the whole table are useful, but the real reports your manager wants are **per department, per city, per customer**. That is GROUP BY. And when the manager says 'only show departments with more than 10 employees', that filter runs after grouping — that is HAVING.

EXPLANATION

> **Expert:** GROUP BY splits the rows into buckets by a column, then runs the aggregate once per bucket. `SELECT department_id, COUNT(*) FROM employees GROUP BY department_id;` produces one row per department with its headcount. HAVING is like WHERE but applies **after** grouping — it filters groups, not raw rows.

CONCEPT CARD

> **Expert:** The clause order and what each does:

1. **FROM** — pick the table.
2. **WHERE** — filter individual rows.
3. **GROUP BY** — bucket the remaining rows.
4. **HAVING** — filter buckets by their aggregate value.
5. **SELECT** — project columns and aggregates.
6. **ORDER BY** — sort the result.
7. **LIMIT** — cap the result.

Rule: every non-aggregate column in SELECT must also appear in GROUP BY.

Employees per department

Scenario. The CTO wants headcount by department for a quarterly review.

Question. How do you count employees in each department?

```
SELECT department_id, COUNT(*) AS headcount
FROM employees
GROUP BY department_id
ORDER BY headcount DESC;
```

Explanation. GROUP BY buckets rows by department_id; COUNT(*) runs per bucket; ORDER BY DESC puts the largest team first.

Employees per city

Scenario. Facilities wants a city-wise headcount to plan office space.

Question. How do you count employees in each city?

```
SELECT city, COUNT(*) AS employees_in_city
FROM employees
GROUP BY city
ORDER BY employees_in_city DESC;
```

Explanation. Same GROUP BY pattern, this time by city. The result is a bar-chart-ready distribution.

Average salary by department

Scenario. HR wants to check pay equity across departments.

Question. How do you compute the average salary per department?

```
SELECT department_id, ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department_id
ORDER BY avg_salary DESC;
```

Explanation. GROUP BY department_id makes AVG run per department. ROUND keeps the report presentation-ready.

Maximum salary by department

Scenario. Which department has the highest single salary?

Question. How do you find the maximum salary within each department?

```
SELECT department_id, MAX(salary) AS top_salary
FROM employees
GROUP BY department_id;
```

Explanation. MAX per group returns each department's highest salary. The row also includes the department it belongs to.

Total salary by department

Scenario. Finance wants the payroll footprint per department for budget allocation.

Question. How do you compute total salary per department?

```
SELECT department_id, SUM(salary) AS payroll
FROM employees
GROUP BY department_id
ORDER BY payroll DESC;
```

Explanation. SUM per group gives the total salary cost per department, sorted from biggest spender to smallest.

Customers per city

Scenario. Marketing wants to know where customers are concentrated.

Question. How do you count customers by city?

```
SELECT city, COUNT(*) AS customers
FROM customers
GROUP BY city
ORDER BY customers DESC;
```

Explanation. GROUP BY city gives one row per city with its customer count. Add LIMIT 10 for top-10 markets.

Departments with more than 10 employees

Scenario. The CTO only cares about medium and large departments — filter out the small ones.

Question. How do you show departments whose headcount is above 10?

```
SELECT department_id, COUNT(*) AS headcount
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 10;
```

Explanation. HAVING applies to the aggregated result. WHERE cannot see COUNT(*), but HAVING can — because HAVING runs after GROUP BY.

Cities with more than 20 customers

Scenario. Marketing wants cities that qualify for a dedicated regional campaign — threshold: 20 customers.

Question. How do you filter cities by customer count?

```
SELECT city, COUNT(*) AS customers
FROM customers
GROUP BY city
HAVING COUNT(*) > 20;
```

Explanation. HAVING filters after grouping. Only cities whose customer count exceeds 20 appear in the result.

Departments with average salary above 70000

Scenario. The board wants to see departments whose average pay exceeds 70k.

Question. How do you show only departments with average salary above 70000?

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 70000
ORDER BY avg_salary DESC;
```

Explanation. HAVING can use any aggregate — AVG here. WHERE cannot, because averages only exist after grouping.

Combine WHERE and HAVING

Scenario. Focus on active employees only (salary > 0) and show departments with more than 5 such employees.

Question. How do you filter rows before grouping AND filter groups after?

```
SELECT department_id, COUNT(*) AS active_employees
FROM employees
WHERE salary > 0
GROUP BY department_id
HAVING COUNT(*) > 5
ORDER BY active_employees DESC;
```

Explanation. WHERE removes zero-salary rows first; GROUP BY buckets the survivors; HAVING keeps only groups with more than 5 members.

Group by two columns

Scenario. You want headcount broken down by both department and city.

Question. How do you group by more than one column?

```
SELECT department_id, city, COUNT(*) AS headcount
FROM employees
GROUP BY department_id, city
ORDER BY department_id, headcount DESC;
```

Explanation. Two columns in GROUP BY means one row per unique (department, city) pair. Perfect for a matrix report.

Orders per customer

Scenario. For a loyalty program, count how many orders each customer has placed.

Question. How do you count orders grouped by customer?

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
ORDER BY order_count DESC;
```

Explanation. One row per customer with their total number of orders — the input to a top-N-customers ranking.

Total order value per customer

Scenario. Sales wants each customer's lifetime spend.

Question. How do you sum orders per customer?

```
SELECT customer_id, SUM(amount) AS lifetime_value
FROM orders
GROUP BY customer_id
ORDER BY lifetime_value DESC;
```

Explanation. SUM per group gives the customer lifetime value (CLV). A LIMIT 10 turns it into a top-10 customers view.

WARNING

> **Expert:** Do not put a non-aggregate column in SELECT that is not in GROUP BY. MySQL under the default `ONLY_FULL_GROUP_BY` mode will reject it. Under older/loose modes it silently returns a random value from the group — a bug factory. Rule: every non-aggregate column in SELECT must also appear in GROUP BY.

COMMON MISTAKE

> **Expert:** Do not filter aggregates with WHERE. `WHERE COUNT(*) > 10` is a syntax error — WHERE runs before GROUP BY, so COUNT does not exist yet. Use HAVING for aggregates. Use WHERE for column values on individual rows.

QUIZ

> **Expert: Q:** Which query lists departments that have more than 5 employees earning above 50000?

A) `SELECT department_id FROM employees WHERE salary > 50000 GROUP BY department_id HAVING COUNT(*) > 5;` B) `SELECT department_id FROM employees GROUP BY department_id HAVING salary > 50000 AND COUNT(*) > 5;` C) `SELECT department_id FROM employees WHERE COUNT(*) > 5 AND salary > 50000 GROUP BY department_id;`

Answer: A. Filter individual rows with WHERE, then filter groups with HAVING.

EXAM TIP CARD

> **Expert:** Interview classic: 'Difference between WHERE and HAVING?' — WHERE filters rows before grouping; HAVING filters groups after grouping. WHERE cannot reference aggregates; HAVING can.

EXERCISE

- > **Expert:** 1. Average, min, and max salary per department in one query.
2. Orders per customer, only those with more than 5 orders.
 3. Cities with average customer spend above 10,000.
 4. Group employees by department and gender; return headcount for each combination.

AI PROMPT

> **Expert:** Try this: 'Write a MySQL query on employees(department_id, salary, gender) that returns one row per department with headcount, average salary, and the male/female ratio, only for departments with more than 20 employees.' Verify the answer includes GROUP BY, HAVING, and correct handling of division.

Figure 3.2 — WHERE vs HAVING - who filters what — A pipeline diagram on a light background flowing left to right. Stage 1 labeled 'FROM employees' shows a stack of ten row icons. Stage 2 labeled 'WHERE salary > 40000' shows a filter funnel with six rows passing through. Stage 3 labeled 'GROUP BY department_id' shows three colored buckets, each holding two rows. Stage 4 labeled 'HAVING COUNT() > 1' shows the same three buckets but two are highlighted green and one is crossed out with red because it did not meet the aggregate condition. Stage 5 labeled 'SELECT department_id, COUNT()' shows the final two output rows. A caption underneath in bold reads 'WHERE filters rows before grouping. HAVING filters groups after grouping.'

CHAPTER REVIEW & SUMMARY

EXPLANATION

> **Expert:** Section 3 taught you how to collapse many rows into summary numbers. Five aggregates — COUNT, SUM, AVG, MIN, MAX — became your toolkit for KPIs and reports. GROUP BY turned every aggregate into a per-bucket report (per department, per city, per customer). HAVING added the ability to filter those buckets by their aggregate values. And you learned the clause execution order: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT. This is the mental model that makes complex reporting queries feel routine.

CONCEPT CARD

> **Expert:** Key takeaways:

- COUNT(*) counts all rows; COUNT(col) counts non-NULLs.
- SUM/AVG/MIN/MAX ignore NULLs.
- GROUP BY buckets rows before aggregation.
- WHERE filters rows; HAVING filters groups.
- Every non-aggregate column in SELECT must appear in GROUP BY.

Next up, in Section 4, you learn to combine multiple tables with JOINS and subqueries — the moment SQL truly becomes powerful.

Section 4 — JOINS, SUBQUERIES AND WINDOW FUNCTIONS

Until now every query touched exactly one table. But real business questions almost always span multiple tables: 'employees with their department names', 'customers with their orders', 'the top 3 salaries per department', 'the second-highest paid employee'. This is where SQL becomes genuinely powerful. Section 4 covers the three techniques that make multi-table analysis possible at ABC Technologies: JOINS (Module 7), subqueries (Module 8), and window functions (Module 9). By the end of this section you can answer virtually any reporting question a manager throws at you.

MODULE 7 - JOINS

A JOIN glues two tables together on a matching column. `employees` has a `department_id`, `departments` has a `department_id` — a JOIN on that column produces a row per employee **with** their department name. This module covers INNER, LEFT, RIGHT, SELF, and CROSS joins through real tickets.

EXPLANATION

> **Expert:** Think of a JOIN as saying: 'for every row in table A, find matching rows in table B based on a shared column, and combine them into a single wider row'. The different types of JOIN differ only in what happens when a row on one side has no match on the other. INNER: drop unmatched rows. LEFT: keep all rows from the left, fill unmatched right side with NULL. RIGHT: mirror image of LEFT. CROSS: every combination. SELF: join a table to itself, useful for employee-manager relations.

CONCEPT CARD

> **Expert:** The five JOIN types you will use at ABC Technologies:

- **INNER JOIN** — only matched rows on both sides.
- **LEFT JOIN** — all rows from left, NULLs on right if no match.
- **RIGHT JOIN** — all rows from right, NULLs on left if no match.
- **SELF JOIN** — join a table to itself (employees to their managers).
- **CROSS JOIN** — Cartesian product; every combination. Rare in practice.

Syntax: `SELECT ... FROM A INNER JOIN B ON A.col = B.col;`

Employees with their department names

Scenario. HR wants a directory that shows every employee alongside their department name, not just the ID.

Question. How do you join employees with departments to show the department name?

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

Explanation. INNER JOIN keeps only employees whose department_id matches a row in departments. Table aliases `e` and `d` keep the query short and readable.

All employees, even those without a department

Scenario. Some new hires may not yet be assigned to a department. The full directory must still show them.

Question. How do you list every employee with department name if it exists?

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
LEFT JOIN departments d
    ON e.department_id = d.department_id;
```

Explanation. LEFT JOIN keeps every employee. Unassigned employees appear with `department_name` as NULL — exactly what the directory needs.

Departments without any employees

Scenario. The CTO suspects a few departments exist on paper but have no one in them. Find them.

Question. How do you find departments with zero employees?

```
SELECT d.department_id, d.department_name
FROM departments d
LEFT JOIN employees e
  ON d.department_id = e.department_id
WHERE e.employee_id IS NULL;
```

Explanation. LEFT JOIN keeps every department; WHERE `e.employee_id IS NULL` keeps only those with no matching employee. Classic 'find orphans' pattern.

Customers with all their orders

Scenario. For a monthly account statement, list every customer next to each of their orders.

Question. How do you join customers to orders?

```
SELECT c.customer_name, o.order_id, o.order_date, o.amount
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
ORDER BY c.customer_name, o.order_date;
```

Explanation. INNER JOIN pairs each customer with every one of their orders. A customer with three orders appears in three rows.

Customers with no orders

Scenario. Sales wants to re-engage customers who have not placed a single order yet.

Question. How do you find customers who have never placed an order?

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Explanation. Same orphan pattern: LEFT JOIN from customers, then keep rows where the orders side is NULL — customers with zero orders.

Orders without a valid customer

Scenario. A data-quality audit wants to find any order whose `customer_id` no longer matches a real customer.

Question. How do you find orphaned orders?

```
SELECT o.order_id, o.customer_id
FROM orders o
LEFT JOIN customers c
  ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
```

Explanation. Mirror the previous pattern with orders on the left. If foreign keys are enforced you should get zero rows — a healthy sign.

Employees with their managers (SELF JOIN)

Scenario. The org chart needs every employee side-by-side with the name of their manager. Both live in the same `employees` table.

Question. How do you join employees with themselves to get manager names?

```
SELECT e.first_name AS employee, m.first_name AS manager
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id;
```

Explanation. A SELF JOIN uses two aliases (`e` and `m`) for the same table. `e.manager_id` points to `m.employee_id`. LEFT JOIN keeps CEOs whose `manager_id` is NULL.

Employees with department and project (three-table JOIN)

Scenario. PMO wants a report: employee name, department, and every project they're on.

Question. How do you join three tables — employees, departments, and projects via `employee_projects`?

```
SELECT e.first_name, d.department_name, p.project_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
INNER JOIN employee_projects ep
    ON e.employee_id = ep.employee_id
INNER JOIN projects p
    ON ep.project_id = p.project_id
ORDER BY e.first_name;
```

Explanation. Chain JOINS by adding more `INNER JOIN ... ON ...` clauses. The many-to-many employees-projects link goes through the `employee_projects` bridge table.

Total order value per customer

Scenario. Sales wants each customer's lifetime spend as a single number next to their name.

Question. How do you join and aggregate at the same time?

```
SELECT c.customer_name, SUM(o.amount) AS lifetime_value
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY lifetime_value DESC;
```

Explanation. JOIN first, then GROUP BY. LEFT JOIN keeps customers with zero orders — their SUM is NULL. Wrap with `COALESCE(SUM(o.amount), 0)` if you prefer 0.

Employees who worked on more than one project

Scenario. PMO wants to spot cross-project contributors.

Question. How do you find employees on multiple projects?

```
SELECT e.first_name, COUNT(ep.project_id) AS projects
FROM employees e
INNER JOIN employee_projects ep
    ON e.employee_id = ep.employee_id
GROUP BY e.employee_id, e.first_name
HAVING COUNT(ep.project_id) > 1;
```

Explanation. Join through the bridge table, group per employee, and filter with HAVING to keep only multi-project contributors.

Total budget managed by each department

Scenario. Finance wants total project budget grouped by department.

Question. How do you sum project budgets per department?

```
SELECT d.department_name, SUM(p.budget) AS total_budget
FROM departments d
LEFT JOIN projects p
  ON d.department_id = p.department_id
GROUP BY d.department_id, d.department_name
ORDER BY total_budget DESC;
```

Explanation. LEFT JOIN keeps departments with no projects (budget will be NULL, which SUM handles as NULL — coalesce if you need 0).

CROSS JOIN example — all shift-city combinations

Scenario. For a scheduling prototype, you want every combination of city × shift. Shifts are inline values.

Question. How do you produce every possible (city, shift) combination?

```
SELECT DISTINCT e.city, s.shift
FROM employees e
CROSS JOIN (
  SELECT 'Morning' AS shift UNION SELECT 'Evening' UNION SELECT 'Night'
) AS s;
```

Explanation. CROSS JOIN produces the Cartesian product — every city paired with every shift. Use sparingly; the row count explodes fast.

WARNING

> **Expert:** Forgetting the `ON` clause on an INNER or LEFT JOIN turns it into an implicit CROSS JOIN — you get every row of A paired with every row of B. On big tables this explodes memory in seconds. Always double-check the ON clause exists and uses the correct keys.

COMMON MISTAKE

> **Expert:** Placing an outer-table filter in the WHERE clause after a LEFT JOIN silently converts it into an INNER JOIN. Example: `LEFT JOIN orders o ON ... WHERE o.amount > 100` — customers with no orders vanish because their `o.amount` is NULL, which fails `> 100`. Move the condition into the ON clause: `LEFT JOIN orders o ON c.customer_id = o.customer_id AND o.amount > 100`.

MEMORY TRICK CARD

> **Expert:** JOIN types by 'who survives with no match': INNER = **nobody**, LEFT = **left side**, RIGHT = **right side**, FULL = **everybody**. Remember: LEFT is the safest default when the left table is the one you cannot afford to lose rows from.

EXERCISE

> **Expert:** 1. Employees, their departments, and their manager's name — all in one query.

2. Customers who have placed at least one order above 10,000.
3. Projects that have no employees assigned via `employee_projects`.
4. Department names alongside average salary of their employees.

Figure 4.1 — The four everyday JOIN patterns — Four Venn diagrams arranged in a 2x2 grid on a light background. Top-left labeled 'INNER JOIN' shows two overlapping circles with only the intersection filled in gold. Top-right labeled 'LEFT JOIN' shows the same two circles with the entire left circle plus the intersection filled in gold. Bottom-left labeled 'RIGHT JOIN' shows the right circle plus intersection filled in gold. Bottom-right labeled 'FULL OUTER JOIN (simulated with UNION)' shows both circles fully filled with a note in small text 'MySQL 8+: emulate with `LEFT JOIN UNION RIGHT JOIN`'. Each diagram is captioned with the SQL keyword and a short ABC Technologies example: INNER = employees WITH departments, LEFT = all employees even if their department is deleted, RIGHT = all departments even if empty, FULL = both. Below the grid a small note reads 'The left circle is the first table named after FROM; the right circle is the second.'

MODULE 8 - SUBQUERIES

A subquery is a query inside another query. When the outer query needs a value that itself has to be computed with SQL — 'employees earning more than the average', 'customers who placed the maximum number of orders' — a subquery is the tool. This module covers scalar subqueries, correlated subqueries, and the classic Nth-highest pattern.

EXPLANATION

> **Expert:** A subquery lives inside parentheses and returns a value (scalar), a list (used with IN), or a whole table (used in FROM). It runs first, then feeds its result into the outer query. Correlated subqueries reference a column from the outer query — they run once per outer row and are slower but often the clearest way to express a problem.

CONCEPT CARD

> **Expert:** The three shapes of subquery:

- **Scalar** — returns one value. `WHERE salary > (SELECT AVG(salary) FROM employees)`.
- **List (IN)** — returns a column of values. `WHERE department_id IN (SELECT department_id FROM departments WHERE ...)`.
- **Derived table (FROM)** — returns a table. `FROM (SELECT ...) AS t`.
- **Correlated** — references the outer query. Slowest but most expressive.

Highest salary using a subquery

Scenario. You want the employee(s) earning the highest salary — the whole row, not just the number.

Question. How do you find the employee earning the highest salary?

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

Explanation. The inner query returns the single MAX value; the outer WHERE keeps rows matching it. Handles ties: if two people earn the max, both appear.

Second-highest salary

Scenario. A classic interview and business question: find the second-highest salary.

Question. How do you get the second-highest salary in the company?

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Explanation. Exclude the top salary in the WHERE clause, then MAX gives the next-highest. Elegant and index-friendly.

Third-highest salary using LIMIT with DISTINCT

Scenario. Generalizing the pattern to Nth-highest with a simple approach.

Question. How do you get the third-highest distinct salary?

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

Explanation. DISTINCT collapses ties into one row, ORDER BY DESC ranks them, OFFSET 2 skips the top two, LIMIT 1 returns the third.

Employees earning above the company average

Scenario. HR wants everyone paid above the company-wide average as a shortlist for high-performer analysis.

Question. How do you find employees earning above the average salary?

```
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Explanation. The scalar subquery computes AVG once; the outer WHERE compares each row to that constant.

Employees earning below the average

Scenario. The mirror image: paid below the average. Used for pay-review planning.

Question. How do you find employees earning less than the average?

```
SELECT first_name, salary
FROM employees
WHERE salary < (SELECT AVG(salary) FROM employees);
```

Explanation. Same pattern flipped: < instead of >. Both queries execute the inner query exactly once.

Customers with the most orders

Scenario. Sales wants to identify the customer(s) who placed the maximum number of orders.

Question. How do you find the customer(s) with the highest order count?

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) = (
    SELECT MAX(oc) FROM (
        SELECT COUNT(*) AS oc FROM orders GROUP BY customer_id
    ) AS t
);
```

Explanation. Nested subquery: the innermost query lists order counts per customer; the middle query gets the MAX of those counts; the outer query keeps customers matching that max.

Products (projects) never assigned to anyone

Scenario. PMO wants projects sitting in the projects table with no one working on them.

Question. How do you find projects that are not in employee_projects?

```
SELECT project_id, project_name
FROM projects
WHERE project_id NOT IN (
    SELECT project_id FROM employee_projects
);
```

Explanation. The subquery returns a list of assigned project IDs. `NOT IN` keeps projects whose id is not in that list. **Caveat:** if the subquery returns any NULL, `NOT IN` returns nothing — filter NULLs first.

Latest order per customer (correlated subquery)

Scenario. For each customer, show their most recent order — a per-row personalized answer.

Question. How do you find each customer's latest order using a correlated subquery?

```

SELECT o.customer_id, o.order_id, o.order_date, o.amount
FROM orders o
WHERE o.order_date = (
    SELECT MAX(o2.order_date)
    FROM orders o2
    WHERE o2.customer_id = o.customer_id
);

```

Explanation. The inner subquery references `o.customer_id` from the outer query, so it re-runs per outer row. Returns each customer's max order_date, then the outer keeps orders matching that date.

Departments whose average salary exceeds the company average

Scenario. The board wants departments paying above the overall company average.

Question. How do you compare a department average against the global average?

```

SELECT department_id, AVG(salary) AS dept_avg
FROM employees
GROUP BY department_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);

```

Explanation. Two averages: the outer per-department average (via GROUP BY) and the scalar subquery for the whole-company average, compared in HAVING.

Employees earning above their department's average

Scenario. HR is doing a within-department comparison: who is above average in their own department?

Question. How do you find employees earning above their department's average?

```

SELECT e.first_name, e.department_id, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department_id = e.department_id
);

```

Explanation. Correlated subquery: the inner query computes the average salary for the same department as the outer row, then the outer WHERE keeps only rows above that per-department average.

Customers who ordered more than the customer average count

Scenario. Loyalty tier: customers whose order count exceeds the average number of orders per customer.

Question. How do you find above-average buyers?

```
SELECT customer_id, COUNT(*) AS orders_placed
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > (
    SELECT AVG(cnt) FROM (
        SELECT COUNT(*) AS cnt FROM orders GROUP BY customer_id
    ) AS per_customer
);
```

Explanation. Inner derived table gives one row per customer with their order count. The middle AVG gives the average across customers. The outer HAVING keeps customers above that average.

WARNING

> **Expert:** WHERE col NOT IN (subquery) returns zero rows if the subquery ever produces a NULL. This burns hours of debugging every year. Either add WHERE col IS NOT NULL to the subquery or switch to NOT EXISTS, which handles NULLs correctly.

COMMON MISTAKE

> **Expert:** Freshers reach for correlated subqueries when a JOIN would be faster. If the inner query does not need to reference the outer row, prefer a JOIN or a derived table. Correlated subqueries re-execute per outer row and can be catastrophic on big tables.

QUIZ

> **Expert: Q:** Which correctly finds the second-highest salary?

A) `SELECT MAX(salary) FROM employees WHERE salary < MAX(salary);` B) `SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);` C) `SELECT salary FROM employees ORDER BY salary DESC LIMIT 2;`

Answer: B. Option A mixes aggregate directly in WHERE which is invalid. Option C returns two salaries, not just the second-highest.

EXAM TIP CARD

> **Expert:** The Nth-highest salary is a top-3 interview question. Memorize both approaches: (1) `DISTINCT salary ... ORDER BY salary DESC LIMIT 1 OFFSET N-1`, and (2) `DENSE_RANK()` (Module 9). Have both in your pocket.

EXERCISE

> **Expert:** 1. Find the 5th-highest distinct salary.

2. Customers whose lifetime spend is above the median customer spend.
3. Employees whose salary equals the maximum salary in their department.
4. Projects with budget above the average project budget.

Figure 4.2 — How a scalar subquery is evaluated — A vertical diagram on a light background showing a two-stage process. Top box labeled 'Step 1: Inner query runs first' contains the SQL `SELECT AVG(salary) FROM employees` with an output arrow producing a single value '65000' inside a gold badge. A downward arrow labeled 'value injected' points into the middle box labeled 'Step 2: Outer query uses the value' which shows `SELECT first_name FROM employees WHERE salary > 65000`. Bottom box labeled 'Result' shows a small table of employees with salaries above 65000. Caption underneath in italics: 'Scalar subqueries execute once and inject their value like a constant.'

MODULE 9 - WINDOW FUNCTIONS

Window functions are the modern SQL upgrade that make rankings, running totals, and per-row analytics elegant. Unlike GROUP BY they do not collapse rows — every row survives with an extra computed column showing its rank, its running total, or its previous/next value.

EXPLANATION

> **Expert:** A window function computes a value for each row over a **window** — a set of related rows defined by an `OVER (PARTITION BY ... ORDER BY ...)` clause. `PARTITION BY` is like `GROUP BY` for the window (per department, per customer). `ORDER BY` inside `OVER` sorts the rows in the window. Unlike `GROUP BY`, the original rows are preserved — the function just adds a new column.

CONCEPT CARD

> **Expert:** The window functions you will use most at ABC Technologies:

- **ROW_NUMBER()** — 1, 2, 3, ... always unique.
- **RANK()** — 1, 2, 2, 4 — gaps after ties.
- **DENSE_RANK()** — 1, 2, 2, 3 — no gaps after ties.
- **SUM(col) OVER (...)** — running or partitioned totals.
- **AVG(col) OVER (...)** — moving averages.
- **LAG(col), LEAD(col)** — previous/next row value.
- **NTILE(n)** — divide rows into n buckets.

Rank employees by salary

Scenario. HR wants a company-wide salary ranking, with ties sharing a rank and the next rank skipping.

Question. How do you rank all employees by salary?

```
SELECT first_name, salary,  
       RANK() OVER (ORDER BY salary DESC) AS salary_rank  
FROM employees;
```

Explanation. `RANK() OVER (ORDER BY salary DESC)` assigns rank 1 to the top salary, ties share the same rank, and the next rank skips accordingly.

Dense rank employees by salary

Scenario. You want ranks without gaps after ties — cleaner for pay-band reports.

Question. How do you produce a dense rank of salaries?

```
SELECT first_name, salary,  
       DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_band  
FROM employees;
```

Explanation. DENSE_RANK also handles ties but does not skip the next rank. Two people tied at rank 2 are followed by rank 3, not rank 4.

Row number employees by salary

Scenario. You want a unique 1..N number per employee, even when salaries tie — useful for deterministic pagination.

Question. How do you number every employee row?

```
SELECT first_name, salary,  
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn  
FROM employees;
```

Explanation. ROW_NUMBER always produces unique consecutive integers, even for ties — the engine breaks them arbitrarily unless you add tiebreaker columns.

Top three salaries per department

Scenario. HR wants the top-3 earners in each department for spot-bonuses.

Question. How do you get the top 3 salaries within each department?

```
SELECT *  
FROM (  
    SELECT first_name, department_id, salary,  
           DENSE_RANK() OVER (  
               PARTITION BY department_id  
               ORDER BY salary DESC  
           ) AS rnk  
    FROM employees  
    ) ranked  
WHERE rnk <= 3;
```

Explanation. PARTITION BY department_id restarts the ranking for each department. Wrap in a subquery so you can filter with `WHERE rnk <= 3` — WHERE cannot reference the window column directly.

Running total of order amounts

Scenario. Finance wants a cumulative revenue chart: for each order date, the total revenue up to and including that date.

Question. How do you compute a running sum of order amounts by date?

```
SELECT order_id, order_date, amount,  
       SUM(amount) OVER (  
         ORDER BY order_date  
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
       ) AS running_total  
FROM orders;
```

Explanation. SUM with an OVER clause that orders by date and includes all rows from the start to the current row produces the classic running total.

Running salary total per department

Scenario. For payroll planning, show the cumulative salary within each department ordered by hire date.

Question. How do you compute a running salary total partitioned by department?

```
SELECT department_id, first_name, hire_date, salary,  
       SUM(salary) OVER (  
         PARTITION BY department_id  
         ORDER BY hire_date  
       ) AS dept_running_total  
FROM employees;
```

Explanation. PARTITION BY resets the running total per department; ORDER BY hire_date controls the accumulation order.

Previous salary using LAG

Scenario. Compare each employee's salary to the person hired just before them, per department.

Question. How do you show the previous row's salary using LAG?

```
SELECT department_id, first_name, hire_date, salary,  
       LAG(salary) OVER (  
         PARTITION BY department_id  
         ORDER BY hire_date  
       ) AS previous_salary  
FROM employees;
```

Explanation. LAG returns the value from the previous row in the window. The first row per department has NULL because there is no previous row.

Next salary using LEAD

Scenario. Show each employee alongside the salary of the person hired just after them in the same department.

Question. How do you show the next row's salary using LEAD?

```
SELECT department_id, first_name, hire_date, salary,  
       LEAD(salary) OVER (  
         PARTITION BY department_id  
         ORDER BY hire_date  
       ) AS next_salary  
FROM employees;
```

Explanation. LEAD is the mirror of LAG — it fetches the next row's value. The last row per department gets NULL.

Salary difference from the previous hire

Scenario. For pay-inflation analysis: how much more does each new hire earn compared to the previous hire in the same department?

Question. How do you compute salary delta versus the previous row?

```
SELECT department_id, first_name, salary,  
       salary - LAG(salary) OVER (  
         PARTITION BY department_id  
         ORDER BY hire_date  
       ) AS delta_from_previous  
FROM employees;
```

Explanation. Subtract LAG(salary) from the current row's salary. Positive means the new hire earns more; negative means less.

Divide employees into 4 salary buckets

Scenario. HR wants everyone bucketed into 4 quartiles by salary for a market study.

Question. How do you place each employee into 1 of 4 salary buckets?

```
SELECT first_name, salary,  
       NTILE(4) OVER (ORDER BY salary DESC) AS quartile  
FROM employees;
```

Explanation. NTILE(4) splits the ordered rows into 4 roughly equal groups. Quartile 1 is the top earners; quartile 4 is the bottom.

WARNING

> **Expert:** Window functions cannot appear in a WHERE clause. To filter by a window result (e.g. 'top 3 per department'), wrap the query in a subquery or CTE and filter in the outer query.

COMMON MISTAKE

> **Expert:** Forgetting PARTITION BY when you want per-group ranking. RANK() OVER (ORDER BY salary DESC) ranks the **whole company**, not the department. Add PARTITION BY department_id inside OVER to restart the rank per department.

MEMORY TRICK CARD

> **Expert:** OVER's two knobs: **PARTITION BY** = 'reset the window here', **ORDER BY** = 'sort inside the window'. Rankings need both; running totals need both; a plain moving sum over the whole table needs neither.

EXAM TIP CARD

> **Expert:** The 'top N per group' question is asked in every SQL interview. Answer template: WITH ranked AS (SELECT ..., DENSE_RANK() OVER (PARTITION BY group_col ORDER BY metric DESC) AS rnk FROM t) SELECT * FROM ranked WHERE rnk <= N;

EXERCISE

> **Expert:** 1. Top 2 customers by lifetime spend within each city.

2. For each employee, the difference between their salary and the department average.
3. Running total of order amount per customer, ordered by order_date.
4. Bucket employees into 5 pay bands and label them 'A' through 'E'.

AI PROMPT

> **Expert:** Try: 'Given an employees table (department_id, salary, hire_date), write a MySQL query that returns each employee with their salary, department average, difference from department average, and their DENSE_RANK within the department by salary desc.' Then verify the answer uses PARTITION BY correctly.

Figure 4.3 — ROW_NUMBER vs RANK vs DENSE_RANK on tied salaries — A 5-row table on a light background with columns: employee, salary, ROW_NUMBER, RANK, DENSE_RANK. Rows in descending salary order: Alice 90000 -> 1, 1, 1. Bob 85000 -> 2, 2, 2. Carol 85000 -> 3, 2, 2. David 80000 -> 4, 4, 3. Eve 75000 -> 5, 5, 4. The tied 85000 salaries (Bob and Carol) are highlighted with a gold border so the reader instantly sees the difference: ROW_NUMBER always increases, RANK skips 3 after the tie, DENSE_RANK does not skip. A caption below explains: 'Use ROW_NUMBER when you need unique labels, RANK when ties should skip positions (like Olympics medals), DENSE_RANK for consecutive rank labels.'

CHAPTER REVIEW & SUMMARY

EXPLANATION

> **Expert:** Section 4 unlocked multi-table SQL. JOINS let you combine rows from different tables — INNER for matched-only, LEFT for keep-all-left, SELF for hierarchies, and three-table chains for reports that touch the whole schema. Subqueries let one query use the result of another — perfect for 'above average', 'Nth highest', and 'per-row personalized' questions. Window functions gave you rankings, running totals, and previous/next lookups without collapsing rows. Together these are the backbone of every dashboard, KPI, and analytical report at ABC Technologies.

CONCEPT CARD

> **Expert:** Key takeaways:

- INNER JOIN drops unmatched, LEFT JOIN preserves the left side.
- SELF JOIN uses two aliases of the same table.
- Scalar subqueries inject a single value; NOT IN + NULL is a footgun.
- Correlated subqueries reference the outer query and run per row.
- Window functions add a computed column per row without collapsing.
- 'Top N per group' = DENSE_RANK() with PARTITION BY, filtered in outer query.

Section 5 shifts from reading data to reshaping the database itself — views, functions, stored procedures, and transactions.

Section 5 — VIEWS, FUNCTIONS, STORED PROCEDURES AND TRANSACTIONS

Until now you have been a consumer of the database — reading, filtering, joining. Section 5 makes you a builder. You will create reusable Views that hide complex queries behind a simple name, write Functions that compute values on demand, package multi-step logic into Stored Procedures, and wrap critical operations in Transactions that either all succeed or all fail. These are the objects that turn a database from a data store into an application platform.

MODULE 10 - VIEWS

A view is a saved SELECT query that you can query like a table. It hides complexity, enforces a consistent shape for reports, and can even restrict which columns a user sees. This module walks you through creating, using, replacing, and dropping views for the reports ABC Technologies runs every week.

EXPLANATION

> **Expert:** A view is a **virtual table**. It stores the SELECT statement, not the data. Every time you query the view, MySQL re-runs the underlying SELECT. Views are perfect when three teams keep writing the same 40-line JOIN — save it once as a view, and everyone queries `SELECT * FROM active_employees` instead.

CONCEPT CARD

> **Expert:** The view lifecycle:

- **CREATE VIEW name AS SELECT ...** — create it.
- **SELECT * FROM name** — use it like a table.
- **CREATE OR REPLACE VIEW name AS ...** — redefine it.

- **DROP VIEW name** — remove it.
- **SHOW CREATE VIEW name** — see how it was defined.

Views can be simple wrappers, JOINS, aggregations, or filters. They cannot store data themselves.

Create a view of active employees

Scenario. Every team keeps re-writing the same JOIN between employees and departments. You will save it once as a view.

Question. How do you create a view that joins employees with their department name?

```
CREATE VIEW active_employees AS
SELECT e.employee_id, e.first_name, e.last_name,
       e.salary, d.department_name, e.city
FROM employees e
INNER JOIN departments d
  ON e.department_id = d.department_id;
```

Explanation. `CREATE VIEW` saves the query under a name. From now on any team can `SELECT * FROM active_employees` instead of repeating the JOIN.

Query the view like a table

Scenario. The Hyderabad office manager wants active employees in her city.

Question. How do you use the active_employees view?

```
SELECT first_name, department_name
FROM active_employees
WHERE city = 'Hyderabad';
```

Explanation. The view behaves exactly like a table — `SELECT`, `WHERE`, `ORDER BY`, and even JOINS all work. MySQL expands the view definition transparently.

Replace the view with a new definition

Scenario. The view needs to include the hire_date column that finance requested last week.

Question. How do you redefine an existing view?

```
CREATE OR REPLACE VIEW active_employees AS
SELECT e.employee_id, e.first_name, e.last_name,
       e.salary, e.hire_date, d.department_name, e.city
FROM employees e
INNER JOIN departments d
  ON e.department_id = d.department_id;
```

Explanation. `CREATE OR REPLACE VIEW` drops and recreates the view atomically. Any existing code that queried it continues to work as long as your column names remain compatible.

Drop a view

Scenario. The `active_employees` view has been replaced by a new star schema and is no longer needed.

Question. How do you delete a view?

```
DROP VIEW active_employees;
```

Explanation. `DROP VIEW` removes the view definition only. The underlying tables are untouched. Use `DROP VIEW IF EXISTS` in scripts to avoid errors when the view may not exist.

Create a department salary report view

Scenario. Finance queries department-wise salary totals weekly. Save the query as a view.

Question. How do you create a view that aggregates salary per department?

```
CREATE OR REPLACE VIEW department_salary_report AS
SELECT d.department_id, d.department_name,
       COUNT(e.employee_id) AS headcount,
       SUM(e.salary)        AS total_salary,
       ROUND(AVG(e.salary), 2) AS avg_salary
FROM departments d
LEFT JOIN employees e
  ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;
```

Explanation. The view precomputes the report's shape. Finance now runs `SELECT * FROM department_salary_report` and gets the up-to-date summary every time.

Create a customer 360 view

Scenario. Sales needs one row per customer with total orders, total revenue, and last order date.

Question. How do you build a customer 360 view combining customers and orders?

```
CREATE OR REPLACE VIEW customer_360 AS
SELECT c.customer_id, c.customer_name, c.city,
       COUNT(o.order_id) AS order_count,
       COALESCE(SUM(o.amount), 0) AS lifetime_value,
       MAX(o.order_date) AS last_order_date
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name, c.city;
```

Explanation. One row per customer. LEFT JOIN keeps customers with zero orders; COALESCE turns their NULL sum into 0 for cleaner reports.

Show the view's underlying definition

Scenario. A new developer inherits the customer_360 view and wants to know how it was written.

Question. How do you retrieve a view's CREATE statement?

```
SHOW CREATE VIEW customer_360;
```

Explanation. Returns the exact CREATE VIEW statement including the SELECT — perfect for reverse-engineering or migrating the view to another environment.

List every view in company_db

Scenario. You want to see every view that exists in the database for cleanup planning.

Question. How do you list all views in company_db?

```
SELECT table_name AS view_name
FROM information_schema.views
WHERE table_schema = 'company_db';
```

Explanation. `information_schema.views` has one row per view. Filtering by `table_schema = 'company_db'` returns only your database's views.

WARNING

> **Expert:** Views do NOT store data. Each query re-runs the underlying SELECT. If the view wraps a slow multi-JOIN, every query against it is slow. When performance matters and data can be slightly stale, use a **materialized** approach: create a real table and refresh it with a scheduled job. MySQL does not have native materialized views like Postgres, so the pattern is 'summary table + refresh procedure' — you will see procedures in Module 12.

COMMON MISTAKE

> **Expert:** Trying to `INSERT INTO` a view that includes `GROUP BY`, `DISTINCT`, or a `JOIN`. Only 'simple' views — a straight `SELECT` from one table with no aggregates — are updatable. For anything more complex, MySQL rejects the write. Treat aggregate views as read-only.

MEMORY TRICK CARD

> **Expert:** Think of a view as a **saved question**. It stores the question (`SELECT`), not the answer (rows). Each time you ask, MySQL re-asks the underlying tables.

Figure 5.1 — How a view is resolved at query time — A three-panel horizontal flow on a light background. Left panel labeled 'User query' shows `SELECT * FROM active_employees WHERE city = 'Hyderabad'`. A large right-pointing arrow labeled 'View expansion' leads to the middle panel labeled 'What MySQL actually runs' showing the fully expanded query with the view's `SELECT` inlined: `SELECT * FROM (SELECT e.first_name, e.city, d.department_name FROM employees e JOIN departments d ON ... WHERE e.status = 'active') AS active_employees WHERE city = 'Hyderabad'`. Another arrow leads to the right panel 'Rows returned' showing 3 employee icons. Caption below reads 'Views store the query, not the data — every `SELECT` re-runs the underlying statement, so keep views on top of indexed columns.'

MODULE 11 - FUNCTIONS

MySQL ships with dozens of built-in functions — `LENGTH`, `UPPER`, `NOW`, `DATE_FORMAT` — and lets you write your own user-defined functions (UDFs). This module covers both: the essential built-ins you will use weekly, and how to create your own reusable functions for tax and bonus computations.

EXPLANATION

> **Expert:** A function takes inputs, computes, and returns one value. Built-ins are already there — string, date, numeric, conditional. User-defined functions let you encapsulate business logic like 'bonus = 10% of salary' so every query calls `bonus(salary)` instead of hard-coding 0.10. Functions can be used anywhere an expression is allowed: SELECT, WHERE, ORDER BY, HAVING.

CONCEPT CARD

> **Expert:** Built-in functions grouped by family:

- **String:** LENGTH, UPPER, LOWER, CONCAT, CONCAT_WS, SUBSTRING, TRIM, REPLACE.
- **Date:** NOW, CURDATE, YEAR, MONTH, DAY, DATEDIFF, DATE_ADD, DATE_FORMAT.
- **Numeric:** ROUND, CEIL, FLOOR, ABS, MOD, POWER.
- **Conditional:** IF, IFNULL, COALESCE, CASE.
- **Aggregate:** COUNT, SUM, AVG, MIN, MAX (from Section 3).

Convert first names to uppercase

Scenario. For a name-badge printout, HR wants first names in ALL CAPS.

Question. How do you uppercase a string column?

```
SELECT UPPER(first_name) AS name_caps
FROM employees;
```

Explanation. `UPPER(col)` returns the string in uppercase. Similarly `LOWER(col)` returns lowercase — the alias keeps the report column readable.

Concatenate first and last name

Scenario. The dashboard needs a single `full_name` column instead of two.

Question. How do you combine two string columns?

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM employees;
```

Explanation. `CONCAT` glues strings together. `CONCAT_WS(' ', first_name, last_name)` is a cleaner alternative when you have a separator — it skips NULLs automatically.

Extract email domain

Scenario. IT wants a report of employees grouped by email domain.

Question. How do you extract the substring after the '@' in an email?

```
SELECT email,  
       SUBSTRING(email, LOCATE('@', email) + 1) AS domain  
FROM employees  
WHERE email IS NOT NULL;
```

Explanation. `LOCATE('@', email)` returns the position of '@'; `SUBSTRING(str, pos)` returns everything from `pos` to the end. Together they extract the domain.

Format hire date as 'Month YYYY'

Scenario. The company anniversary email needs 'March 2019' formatting for each hire.

Question. How do you format a date column?

```
SELECT first_name,  
       DATE_FORMAT(hire_date, '%M %Y') AS hire_month  
FROM employees;
```

Explanation. `DATE_FORMAT` takes a date and a format string. `%M` = full month name, `%Y` = four-digit year. See the MySQL docs for the full list of format specifiers.

Employees tenure in years

Scenario. For long-service awards, compute how many years each employee has worked.

Question. How do you calculate years between `hire_date` and today?

```
SELECT first_name,  
       FLOOR(DATEDIFF(CURDATE(), hire_date) / 365) AS years_of_service  
FROM employees;
```

Explanation. `DATEDIFF(a, b)` returns the day difference; dividing by 365 approximates years; `FLOOR` discards partial years. For exact years use `TIMESTAMPDIFF(YEAR, hire_date, CURDATE())`.

Round salary to nearest 1000

Scenario. The pay-band report wants salaries rounded to the nearest thousand.

Question. How do you round to the nearest thousand?

```
SELECT first_name, salary,  
       ROUND(salary, -3) AS rounded_salary  
FROM employees;
```

Explanation. `ROUND(n, d)` with negative `d` rounds to the left of the decimal — `-3` rounds to the nearest thousand.

Replace NULL email with a placeholder using IFNULL

Scenario. For a printable directory, blank emails must show 'no email on file'.

Question. How do you substitute a value for NULL?

```
SELECT first_name,  
       IFNULL(email, 'no email on file') AS display_email  
FROM employees;
```

Explanation. `IFNULL(expr, alt)` returns `alt` if `expr` is NULL, else `expr`. Perfect for presentation-friendly output.

Pick the first non-null value with COALESCE

Scenario. Employees may have a personal phone, a work phone, or nothing. Show whichever is available.

Question. How do you pick the first non-NULL value from multiple columns?

```
SELECT first_name,  
       COALESCE(phone, email, 'no contact') AS best_contact  
FROM employees;
```

Explanation. `COALESCE` walks left to right and returns the first non-NULL argument. Same as chained IFNULLs but cleaner.

Create a bonus function

Scenario. The company policy is: bonus = 10% of salary. You will save this as a reusable function.

Question. How do you create a user-defined function that returns bonus?

```
DELIMITER $$
CREATE FUNCTION calculate_bonus(sal DECIMAL(10, 2))
RETURNS DECIMAL(10, 2)
DETERMINISTIC
BEGIN
    RETURN sal * 0.10;
END $$
DELIMITER ;
```

Explanation. `DELIMITER $$` lets you use semicolons inside the function body without MySQL treating them as statement terminators. `DETERMINISTIC` marks the function safe for replication.

Use the bonus function

Scenario. Now compute every employee's bonus.

Question. How do you call your bonus function?

```
SELECT first_name, salary,
       calculate_bonus(salary) AS bonus
FROM employees;
```

Explanation. User-defined functions work exactly like built-ins — call them by name with parentheses. Cleaner than repeating `salary * 0.10` across queries.

Create a tax function

Scenario. Tax is 20% for salary above 50000, otherwise 10%. Encapsulate this rule.

Question. How do you create a function with conditional logic?

```

DELIMITER $$
CREATE FUNCTION calculate_tax(sal DECIMAL(10, 2))
RETURNS DECIMAL(10, 2)
DETERMINISTIC
BEGIN
    IF sal > 50000 THEN
        RETURN sal * 0.20;
    ELSE
        RETURN sal * 0.10;
    END IF;
END $$
DELIMITER ;

```

Explanation. Inside a function body you have full procedural SQL: IF/ELSE, loops, variables. The RETURN statement exits with the final value.

Use bonus and tax in one report

Scenario. Finance wants a single report showing salary, bonus, tax, and net pay.

Question. How do you combine multiple functions in one SELECT?

```

SELECT first_name, salary,
       calculate_bonus(salary) AS bonus,
       calculate_tax(salary)   AS tax,
       salary + calculate_bonus(salary) - calculate_tax(salary) AS net_pay
FROM employees;

```

Explanation. Functions compose freely. Save common formulas once and every report reads cleanly.

Drop a function

Scenario. The tax rule changed and you will replace the function next; first drop the old one.

Question. How do you delete a function?

```

DROP FUNCTION IF EXISTS calculate_tax;

```

Explanation. DROP FUNCTION IF EXISTS removes the function safely — no error if it does not exist. Recreate it right after.

WARNING

> **Expert:** User-defined functions inside WHERE clauses often prevent index usage. `WHERE calculate_bonus(salary) > 5000` cannot use an index on salary because MySQL cannot know how the function transforms it. Rewrite as `WHERE salary > 50000` when possible.

COMMON MISTAKE

> **Expert:** Forgetting `DETERMINISTIC` or `NO SQL / READS SQL DATA` when creating a function under strict binary-log settings — MySQL will refuse the CREATE. If the function always returns the same value for the same input, mark it `DETERMINISTIC`.

MEMORY TRICK CARD

> **Expert:** Functions **return one value**. Procedures (next module) **do actions** and can return zero, one, or many result sets. Rule of thumb: 'need a value in an expression' -> function; 'need a task performed' -> procedure.

Figure 5.2 — User-defined function anatomy — A vertical breakdown diagram on a light background. Top box titled 'CREATE FUNCTION' shows the header: `CREATE FUNCTION calculate_bonus(sal DECIMAL(10,2)) RETURNS DECIMAL(10,2)`. Middle box titled 'DETERMINISTIC clause' shows `DETERMINISTIC` and a small explanation 'same input -> same output; required in strict mode'. Next box titled 'BEGIN ... END body' shows a placeholder `BEGIN DECLARE bonus DECIMAL(10,2); SET bonus = sal * 0.10; RETURN bonus; END`. Bottom box titled 'CALL SITE' shows `SELECT first_name, calculate_bonus(salary) FROM employees`. Down the right edge a callout box reads 'DELIMITER \$\$... \$\$ DELIMITER ; required in mysql client because the function body contains semicolons.'

MODULE 12 - STORED PROCEDURES

A stored procedure is a named block of SQL you can call. Unlike functions, procedures can perform multi-step tasks: insert rows, update salaries, produce report sets, and even control transactions. This module walks you through creating and calling procedures for common ABC Technologies operations.

EXPLANATION

> **Expert:** Where a function returns one value inside an expression, a procedure runs a **block** of statements when you `CALL` it. Procedures take `IN`, `OUT`, and `INOUT` parameters, can produce result sets, and are the natural place to package 'insert employee', 'update salary', or 'monthly close' logic.

CONCEPT CARD

> **Expert:** Procedure basics:

- **CREATE PROCEDURE** `name(param1, param2) BEGIN ... END` — define.
- **CALL** `name(arg1, arg2)` — execute.
- **DROP PROCEDURE** `name` — remove.
- Parameter modes: **IN** (default), **OUT** (write back), **INOUT** (both).
- Use **DELIMITER \$\$** to change the statement delimiter while defining the body.

Create a procedure that returns an employee by ID

Scenario. Support wants a `get_employee` call that returns the full row for a given ID.

Question. How do you create a procedure that takes an ID and returns a row?

```
DELIMITER $$
CREATE PROCEDURE get_employee(IN p_id INT)
BEGIN
    SELECT * FROM employees WHERE employee_id = p_id;
END $$
DELIMITER ;
```

Explanation. The procedure body is one `SELECT`. When you `CALL` it with an ID, MySQL returns the matching row as a result set.

Call the `get_employee` procedure

Scenario. Fetch employee with ID 42.

Question. How do you invoke a stored procedure?

```
CALL get_employee(42);
```

Explanation. `CALL name(args)` runs the procedure and returns whatever result set(s) it produces. Works from any client, ORM, or JDBC driver.

Create a procedure to raise a salary

Scenario. HR gives annual raises via a repeatable procedure so no one has to remember the exact UPDATE syntax.

Question. How do you create a procedure that updates one employee's salary?

```
DELIMITER $$
CREATE PROCEDURE update_salary(
  IN p_id INT,
  IN p_new_salary DECIMAL(10, 2)
)
BEGIN
  UPDATE employees
  SET salary = p_new_salary
  WHERE employee_id = p_id;
END $$
DELIMITER ;
```

Explanation. Procedure signatures make the operation self-documenting. Call it with `CALL update_salary(42, 95000);` and you cannot forget the WHERE clause.

Create a procedure to insert a new employee

Scenario. Onboarding wants a one-call helper that adds a new employee with sane defaults.

Question. How do you create an insert_employee procedure?

```

DELIMITER $$
CREATE PROCEDURE insert_employee(
  IN p_first_name VARCHAR(50),
  IN p_last_name  VARCHAR(50),
  IN p_salary     DECIMAL(10, 2),
  IN p_dept_id    INT
)
BEGIN
  INSERT INTO employees(first_name, last_name, salary,
                        department_id, hire_date)
  VALUES (p_first_name, p_last_name, p_salary,
          p_dept_id, CURDATE());
END $$
DELIMITER ;

```

Explanation. The procedure encapsulates the INSERT, the column list, and the CURDATE() default. Onboarding calls it in one line and cannot forget hire_date.

Create a procedure to delete an employee

Scenario. Offboarding runs a `delete_employee` call rather than raw DELETE — safer and auditable.

Question. How do you create a delete_employee procedure?

```

DELIMITER $$
CREATE PROCEDURE delete_employee(IN p_id INT)
BEGIN
  DELETE FROM employees WHERE employee_id = p_id;
END $$
DELIMITER ;

```

Explanation. Wrapping DELETE in a procedure gives you a single place to add audit logging or soft-delete logic later without changing every caller.

Procedure with an OUT parameter

Scenario. Ops wants to compute the total salary paid by a department and get it back as a variable, not a result set.

Question. How do you use an OUT parameter?

```

DELIMITER $$
CREATE PROCEDURE department_payroll(
    IN p_dept_id INT,
    OUT p_total DECIMAL(12, 2)
)
BEGIN
    SELECT SUM(salary) INTO p_total
    FROM employees
    WHERE department_id = p_dept_id;
END $$
DELIMITER ;

```

Explanation. `SELECT ... INTO variable` writes the aggregate into the OUT parameter. Call with `CALL department_payroll(2, @t); SELECT @t;`

List all procedures in company_db

Scenario. During cleanup you want to see every procedure defined in the database.

Question. How do you list stored procedures?

```

SELECT routine_name
FROM information_schema.routines
WHERE routine_schema = 'company_db'
    AND routine_type = 'PROCEDURE';

```

Explanation. `information_schema.routines` catalogs both functions and procedures. Filter `routine_type = 'PROCEDURE'` for procedures only.

WARNING

> **Expert:** Procedures run with definer's privileges by default. That means anyone who can `CALL` a procedure can trigger `UPDATES` and `DELETES` they might not normally have permission to run. Grant `EXECUTE` on procedures carefully, and read the `SQL SECURITY` clause of `CREATE PROCEDURE` for tighter control.

COMMON MISTAKE

> **Expert:** Forgetting to change the `DELIMITER` when creating a procedure with multiple statements in the body. Without `DELIMITER $$`, the first semicolon inside the body ends the `CREATE` statement prematurely and MySQL errors out.

MEMORY TRICK CARD

> **Expert:** CREATE FUNCTION -> returns a value -> used in expressions. CREATE PROCEDURE -> does a job -> called with CALL. The 'C' in CALL matches the 'C' in 'proCedure'.

Figure 5.3 — Function vs Procedure at a glance — A two-column comparison table on a light background. Left column header 'Function' lists five properties: returns a single value; used inside expressions like SELECT and WHERE; parameters IN only; cannot easily emit result sets; typically DETERMINISTIC. Right column header 'Procedure' lists: runs a block of statements; called with the CALL keyword; parameters IN, OUT, or INOUT; can return zero or many result sets; can execute DML and control transactions. Between them a vertical divider labeled 'return one value <--> do one task'. Below the table a colored footer with an ABC Technologies example on each side: `SELECT calculate_bonus(salary) FROM employees;` for the function side, `CALL insert_employee('Ravi', 'Kumar', 55000);` for the procedure side.

MODULE 13 - TRANSACTIONS

A transaction is a group of SQL statements that must either **all** succeed or **all** fail. Transfer money from account A to B: debit A, credit B — if either step fails, both must roll back. This module covers COMMIT, ROLLBACK, and SAVEPOINT through real ABC Technologies operations like payment processing and inventory updates.

EXPLANATION

> **Expert:** By default MySQL runs in **autocommit** mode — every statement is its own transaction, immediately permanent. When you need multiple statements to succeed or fail as one unit, you start a transaction with `START TRANSACTION`, run the statements, and finish with either `COMMIT` (make permanent) or `ROLLBACK` (undo everything). This is the foundation of the **ACID** guarantees: Atomicity, Consistency, Isolation, Durability.

CONCEPT CARD

> **Expert:** The transaction commands:

- **START TRANSACTION** or **BEGIN** — open a transaction.
- **COMMIT** — make all changes permanent.
- **ROLLBACK** — undo all changes since the transaction started.
- **SAVEPOINT name** — mark a point to roll back to.
- **ROLLBACK TO SAVEPOINT name** — undo only back to that point.
- **RELEASE SAVEPOINT name** — discard a savepoint.

- **SET autocommit = 0** — disable autocommit for the session.

Basic transaction: transfer money between accounts

Scenario. Two employees have prepaid meal-card accounts. Move 500 rupees from Alice's card to Bob's card. Both updates must succeed or both must fail.

Question. How do you group two updates into a single atomic transaction?

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE holder = 'Alice';
UPDATE accounts SET balance = balance + 500 WHERE holder = 'Bob';
COMMIT;
```

Explanation. START TRANSACTION opens the batch; the two UPDATES are staged; COMMIT makes them permanent atomically. If your session crashes between them, MySQL rolls back automatically.

Roll back a bad update

Scenario. You accidentally ran `UPDATE employees SET salary = 0` without a WHERE clause inside a transaction. Undo it.

Question. How do you undo everything since the transaction started?

```
START TRANSACTION;
UPDATE employees SET salary = 0;    -- disaster
ROLLBACK;                          -- undo the disaster
-- verify
SELECT COUNT(*) FROM employees WHERE salary = 0;
```

Explanation. ROLLBACK reverts all changes since the last START TRANSACTION. This is why you always run destructive changes inside a transaction — it lets you check first and only COMMIT if the result looks right.

Use a SAVEPOINT for partial rollback

Scenario. You are running a batch of 3 updates. If the third fails, you want to keep the first two but undo only the third.

Question. How do you mark a savepoint and roll back to it?

```
START TRANSACTION;
UPDATE employees SET salary = salary * 1.05 WHERE department_id = 1;
UPDATE employees SET salary = salary * 1.05 WHERE department_id = 2;
SAVEPOINT before_dept3;
UPDATE employees SET salary = salary * 1.05 WHERE department_id = 3;
-- realized dept 3 already got a raise last week
ROLLBACK TO SAVEPOINT before_dept3;
COMMIT;
```

Explanation. SAVEPOINT marks a checkpoint; ROLLBACK TO SAVEPOINT undoes only the statements executed after that mark; COMMIT then makes the surviving changes permanent.

Online-shopping payment as a transaction

Scenario. A customer pays for an order. You must: insert the order, decrement inventory, and record the payment — all or none.

Question. How do you wrap multi-table writes in one transaction?

```
START TRANSACTION;
INSERT INTO orders(customer_id, order_date, amount)
VALUES (101, CURDATE(), 2500);
UPDATE inventory SET stock = stock - 1 WHERE sku = 'SKU-77';
INSERT INTO payments(order_id, amount, status)
VALUES (LAST_INSERT_ID(), 2500, 'CAPTURED');
COMMIT;
```

Explanation. Three tables, one atomic write. `LAST_INSERT_ID()` returns the ID from the just-inserted order so the payment can reference it. If any step fails, ROLLBACK undoes all three.

Inventory update with rollback on failure

Scenario. You are decrementing stock. If any item goes negative, roll back everything.

Question. How do you validate mid-transaction and roll back if a check fails?

```

START TRANSACTION;
UPDATE inventory SET stock = stock - 5 WHERE sku = 'SKU-77';
SELECT stock INTO @s FROM inventory WHERE sku = 'SKU-77';
-- if stock went negative, undo
-- run this only if @s >= 0
-- otherwise:
-- ROLLBACK;
COMMIT;

```

Explanation. Read the mid-state into a variable, and let the application decide to COMMIT or ROLLBACK. For a fully server-side version, wrap the logic in a stored procedure with IF/ROLLBACK inside.

Disable autocommit for a session

Scenario. For a bulk-load session you want to control commits manually rather than wrap each statement in START TRANSACTION.

Question. How do you turn off autocommit?

```

SET autocommit = 0;
-- run many statements
INSERT INTO customers(customer_name, city) VALUES ('C1', 'Hyderabad');
INSERT INTO customers(customer_name, city) VALUES ('C2', 'Chennai');
COMMIT;
SET autocommit = 1;

```

Explanation. With `autocommit = 0`, MySQL waits for an explicit COMMIT or ROLLBACK. Perfect for load scripts. Restore autocommit at the end so the session behaves normally afterwards.

Wrap a stored procedure in a transaction

Scenario. The `insert_employee` procedure also needs to log to an audit table. Both must succeed together.

Question. How do you use a transaction inside a procedure?

```

DELIMITER $$
CREATE PROCEDURE insert_employee_audited(
    IN p_first VARCHAR(50),
    IN p_last  VARCHAR(50),
    IN p_sal   DECIMAL(10,2),
    IN p_dept  INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;
    START TRANSACTION;
    INSERT INTO employees(first_name, last_name, salary, department_id, hire_date)
    VALUES (p_first, p_last, p_sal, p_dept, CURDATE());
    INSERT INTO audit_log(action, ts) VALUES ('new employee', NOW());
    COMMIT;
END $$
DELIMITER ;

```

Explanation. `DECLARE EXIT HANDLER FOR SQLEXCEPTION` catches any error, rolls back, and re-raises. The result: either both inserts succeed or neither does.

WARNING

> **Expert:** DDL statements (`CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`) implicitly `COMMIT` the current transaction. If you `START TRANSACTION`, run some updates, then run `ALTER TABLE`, MySQL will commit your updates automatically — no way to roll back after that. Keep schema changes out of business transactions.

COMMON MISTAKE

> **Expert:** Assuming MyISAM tables support transactions. MyISAM does NOT — it silently ignores `START TRANSACTION` and `COMMIT/ROLLBACK`. Use InnoDB (the default since MySQL 5.5) for anything that needs transactional guarantees.

MEMORY TRICK CARD

> **Expert:** ACID: **A**tomic (all or nothing), **C**onsistent (respects constraints), **I**solated (concurrent transactions don't see each other's uncommitted work), **D**urable (once committed, survives a crash). Every transaction question in an interview traces back to one of these four.

QUIZ

> **Expert: Q:** You issue START TRANSACTION, two UPDATES, then ALTER TABLE, then ROLLBACK. What is the final state?

A) Both UPDATES and the ALTER are undone. B) Both UPDATES succeed (committed by the ALTER), ALTER succeeds. C) Everything succeeds despite the ROLLBACK.

Answer: B. The ALTER implicitly commits, so the ROLLBACK finds nothing to undo. Never mix DDL into a business transaction.

EXAM TIP CARD

> **Expert:** Interview classic: 'Explain ACID with a bank-transfer example.' Answer: debit A and credit B must be Atomic (both or none), maintain Consistent balances (no money vanishes), be Isolated from concurrent transfers, and Durable once committed. Practice saying it in under 45 seconds.

EXERCISE

> **Expert:** 1. Wrap the online-shopping payment in a transaction that rolls back if inventory goes negative.

2. Create a `promote_employee` procedure that raises salary AND writes an audit log, atomically.
3. Use a SAVEPOINT to safely try a bulk update on a subset of employees before committing.
4. Explain in one sentence why DDL should not be mixed with transactions.

Figure 5.4 — Money transfer as a transaction — A vertical flowchart on a light background showing a money-transfer scenario. Start block labeled 'START TRANSACTION' in green. First action box: 'UPDATE accounts SET balance = balance - 1000 WHERE id = A' with a rupee symbol and a small green check mark. Second action box: 'UPDATE accounts SET balance = balance + 1000 WHERE id = B' also with a check. A decision diamond asks 'All steps OK?' with

two branches: 'Yes' arrow leads to a green COMMIT terminator labeled 'Money moved, both accounts updated permanently'; 'No' arrow leads to a red ROLLBACK terminator labeled 'Both updates undone, balances unchanged'. On the right side a labeled callout shows ACID: A=Atomic, C=Consistent, I=Isolated, D=Durable — with a short one-line explanation for each.

CHAPTER REVIEW & SUMMARY

EXPLANATION

> **Expert:** Section 5 turned you from a reader into a builder. Views hid your complex queries behind a friendly name so every team runs `SELECT * FROM customer_360` instead of reinventing the JOIN. Functions packaged reusable formulas — bonus, tax, tenure — into named calls that fit into any SELECT. Procedures wrapped multi-step operations like `insert_employee` or `update_salary` so every caller runs the same audited logic. And transactions gave you the safety net of ACID: either all statements succeed or none do, no half-updated rows left behind. Together, these four objects are how a schema grows into an application platform.

CONCEPT CARD

> **Expert:** Key takeaways:

- Views are saved queries, not saved data — refactor SQL, not storage.
- Built-in functions cover string, date, numeric, and conditional needs.
- User-defined functions encapsulate business rules that live in expressions.
- Procedures package multi-step tasks called with `CALL`.
- Transactions turn many statements into one atomic unit.
- ACID guarantees rely on InnoDB — do not use MyISAM for anything that matters.

Section 6 finishes the Refcard with the operational side: indexes, metadata, and performance tuning.

Section 6 — INDEXES, METADATA AND PERFORMANCE

You have written every kind of query — retrieval, filtering, aggregation, joins, subqueries, window functions, procedures. Now your manager gives you the final skill of a real database engineer: make them **fast**. Slow reports are the number-one complaint in every company. Section 6 covers the three tools that separate seniors from juniors: indexes (Module 14), metadata (Module 15),

and performance tuning with EXPLAIN (Module 16). By the end you will not just write queries — you will make them scale.

MODULE 14 - INDEXES

An index is a data structure that lets MySQL find rows without scanning the whole table. Think of the index in a book: if you want the chapter on JOINS, you flip to the index instead of reading every page. Same idea. This module teaches you when and how to create indexes for real ABC Technologies workloads.

EXPLANATION

> **Expert:** Without an index, a query like `WHERE email = 'x@y.com'` performs a **full table scan** — every row is read from disk and checked. With an index on `email`, MySQL walks a B-tree in a handful of hops and jumps straight to the row. On a 10-million-row table that is the difference between 10 seconds and 1 millisecond. Indexes speed reads dramatically but slow writes slightly (every INSERT/UPDATE also updates the index), and they consume disk space. Rule: index the columns you filter, join, or sort on most often.

CONCEPT CARD

> **Expert:** Index vocabulary at ABC Technologies:

- **Primary key index** — automatically created for the PRIMARY KEY.
- **Unique index** — like a normal index but also enforces uniqueness.
- **Non-unique index** — the everyday index for fast lookups.
- **Composite index** — over multiple columns, e.g. (city, salary).
- **Covering index** — an index that contains all columns needed by a query.
- **B-tree** — the default MySQL index structure.
- **Full-text index** — special index for `MATCH ... AGAINST` text search.

Create an index on email

Scenario. The support team searches employees by email hundreds of times a day. The search is slow.

Question. How do you create an index on the email column?

```
CREATE INDEX idx_employees_email  
ON employees(email);
```

Explanation. `CREATE INDEX name ON table(column)` builds a B-tree on that column. Every `WHERE email = ...` now uses the index and returns in milliseconds instead of seconds.

Create a unique index on email

Scenario. Business rule: no two employees share an email. Enforce it at the database level.

Question. How do you create a unique index that also enforces uniqueness?

```
CREATE UNIQUE INDEX uq_employees_email  
ON employees(email);
```

Explanation. `CREATE UNIQUE INDEX` adds both the index and a uniqueness constraint. Any future `INSERT` of a duplicate email fails with an error.

Create a composite index on city and salary

Scenario. The HR dashboard filters by city then sorts by salary. A composite index will cover both operations.

Question. How do you create an index across two columns?

```
CREATE INDEX idx_employees_city_salary  
ON employees(city, salary);
```

Explanation. A composite index on (city, salary) makes `WHERE city = 'X' ORDER BY salary` index-only. Column order matters: put the equality-filtered column first, the range/sort column second.

List all indexes on employees

Scenario. You inherit the employees table and want to see what indexes exist.

Question. How do you list every index on a table?

```
SHOW INDEX FROM employees;
```

Explanation. `SHOW INDEX` returns one row per index-column pair with key name, seq_in_index, cardinality, and index type — everything needed to audit the indexing strategy.

Drop an unused index

Scenario. An old index on `phone` is no longer used — remove it to speed up writes.

Question. How do you delete an index?

```
DROP INDEX idx_employees_phone ON employees;
```

Explanation. `DROP INDEX name ON table` removes the index. The table data is untouched — only the auxiliary structure disappears.

Add an index using ALTER TABLE

Scenario. Some teams prefer ALTER TABLE syntax to keep DDL uniform.

Question. How do you add an index via ALTER TABLE?

```
ALTER TABLE orders  
ADD INDEX idx_orders_customer_date (customer_id, order_date);
```

Explanation. `ALTER TABLE ... ADD INDEX` is functionally identical to `CREATE INDEX`. Choose whichever style your team standardizes on.

Foreign key with an implicit index

Scenario. You add a foreign key on `orders.customer_id`. MySQL auto-creates an index.

Question. How do you add a foreign key that also creates an index?

```
ALTER TABLE orders  
ADD CONSTRAINT fk_orders_customer  
    FOREIGN KEY (customer_id)  
    REFERENCES customers(customer_id);
```

Explanation. InnoDB requires an index on the FK column. If none exists, MySQL creates one automatically. This is why FK columns are usually already fast to join on.

Show which index a query used

Scenario. You added an index on email but a query still feels slow. Verify MySQL is actually using the index.

Question. How do you check which index a query uses?

```
EXPLAIN  
SELECT * FROM employees WHERE email = 'ravi@company.com';
```

Explanation. `EXPLAIN` prefixes any `SELECT` and returns MySQL's execution plan. Look at the `key` column — if it names your index, you're using it. If it shows `NULL`, MySQL did a full scan.

WARNING

> **Expert:** Do NOT create an index on every column 'just in case'. Every index slows down INSERT/UPDATE/DELETE because it must be maintained on every write. Index the columns you actually query on WHERE, JOIN, or ORDER BY. Measure with EXPLAIN before adding, and drop indexes you never use.

COMMON MISTAKE

> **Expert:** Wrapping an indexed column in a function inside WHERE. `WHERE UPPER(email) = 'RAVI@X.COM'` cannot use the index on email — the function transforms the column before comparison. Either store the column already normalized or create a functional index (`CREATE INDEX ... ON employees((UPPER(email)))`).

MEMORY TRICK CARD

> **Expert:** The index-column order rule: **Equality columns first, then range/sort**. For `WHERE city = 'X' AND salary > 50000 ORDER BY hire_date`, the best composite index is `(city, salary, hire_date)` — but often just `(city, salary)` is enough.

EXAM TIP CARD

> **Expert:** Interview classic: 'When would you NOT use an index?' Answer: on very small tables (a full scan is faster), on columns with very low cardinality (like gender with 2 values), and on write-heavy columns where the maintenance cost outweighs the read benefit.

Figure 6.1 — Full table scan vs index lookup — A two-panel side-by-side illustration on a light background. Left panel labeled 'Without index' shows a stack of 10 employee row rectangles with a large magnifying glass sweeping through every single one; each is marked with a red check to indicate 'read from disk'. A caption ' $O(n)$ - full table scan' sits below. Right panel labeled 'With index on email' shows a small B-tree with three levels: root node at top, three internal nodes in the middle, six leaf nodes at the bottom. A gold arrow drops from root to one specific leaf then points to a single highlighted employee row. Caption ' $O(\log n)$ - index lookup' sits below. Between the two panels a large VS symbol. At the bottom, a performance graph shows scan time going up steeply as row count grows, while index time stays nearly flat.

MODULE 15 - METADATA

Metadata is 'data about the data'. Everything you have created — tables, columns, keys, indexes, views, procedures, functions — is catalogued inside `INFORMATION_SCHEMA`. This module gives you the queries that answer 'what exists in my database?' programmatically.

EXPLANATION

> **Expert:** `INFORMATION_SCHEMA` is a set of read-only views that expose the internal catalog of MySQL. Every question of the form 'what tables exist?', 'which columns has a nullable state?', 'what foreign keys point to customers?' is answered with a `SELECT` against `INFORMATION_SCHEMA`. It is standardized SQL — the same queries work on many databases.

CONCEPT CARD

> **Expert:** The most useful `INFORMATION_SCHEMA` views:

- **schemata** — databases.
- **tables** — tables and views.
- **columns** — every column in every table.
- **statistics** — indexes.
- **key_column_usage** — primary and foreign keys.
- **table_constraints** — all constraints.
- **views** — views.
- **routines** — functions and procedures.
- **triggers** — triggers.

List all databases

Scenario. You want a programmatic list of databases for a shell script.

Question. How do you query the databases from metadata?

```
SELECT schema_name
FROM information_schema.schemata;
```

Explanation. `information_schema.schemata` has one row per database. Use it whenever you need the list programmatically — `SHOW DATABASES` is fine at the `mysql` prompt but not from a query.

List all tables in company_db

Scenario. You want a queryable list of tables for a report.

Question. How do you list every table in company_db?

```
SELECT table_name, table_type
FROM information_schema.tables
WHERE table_schema = 'company_db';
```

Explanation. `table_type` distinguishes 'BASE TABLE' from 'VIEW'. Filter on it to get only one kind.

List every column in employees

Scenario. Generate documentation: column name, data type, nullability.

Question. How do you list all columns of a specific table?

```
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'company_db'
      AND table_name = 'employees'
ORDER BY ordinal_position;
```

Explanation. `ordinal_position` preserves the column order from the CREATE TABLE. Perfect input for auto-generated documentation or data dictionaries.

Find the primary key columns of every table

Scenario. You want a report of primary keys across the whole database.

Question. How do you find primary key columns using metadata?

```
SELECT table_name, column_name
FROM information_schema.key_column_usage
WHERE table_schema = 'company_db'
      AND constraint_name = 'PRIMARY';
```

Explanation. `key_column_usage` catalogs all key columns. `constraint_name = 'PRIMARY'` filters to primary keys only.

Find all foreign keys

Scenario. Generate a diagram of every FK relationship in company_db.

Question. How do you list foreign keys?

```
SELECT table_name, column_name,  
       referenced_table_name, referenced_column_name  
FROM information_schema.key_column_usage  
WHERE table_schema = 'company_db'  
AND referenced_table_name IS NOT NULL;
```

Explanation. FK rows have a non-NULL `referenced_table_name`. Filtering on that returns just the relationships — perfect for ER-diagram generation.

Find unique constraints

Scenario. You want to list every UNIQUE constraint defined in the database.

Question. How do you list unique constraints from metadata?

```
SELECT table_name, constraint_name  
FROM information_schema.table_constraints  
WHERE table_schema = 'company_db'  
AND constraint_type = 'UNIQUE';
```

Explanation. `table_constraints` catalogs PRIMARY KEY, UNIQUE, FOREIGN KEY, and CHECK constraints. Filter by `constraint_type` for the kind you want.

List all views

Scenario. You want to see every view definition in the database.

Question. How do you list views with their SELECT statements?

```
SELECT table_name AS view_name, view_definition  
FROM information_schema.views  
WHERE table_schema = 'company_db';
```

Explanation. `view_definition` contains the SELECT that the view represents — handy for batch review or export.

List all procedures and functions

Scenario. Get an inventory of all routines defined in `company_db`.

Question. How do you list routines with their type?

```
SELECT routine_name, routine_type, data_type
FROM information_schema.routines
WHERE routine_schema = 'company_db';
```

Explanation. `routine_type` is 'PROCEDURE' or 'FUNCTION'. `data_type` is meaningful for functions (their return type). Both together are your routines audit.

Table size in MB

Scenario. Same query as Module 2 but now presented as a metadata pattern for reference.

Question. How do you compute table size in MB from metadata?

```
SELECT table_name,
       ROUND((data_length + index_length) / 1024 / 1024, 2) AS size_mb
FROM information_schema.tables
WHERE table_schema = 'company_db'
ORDER BY size_mb DESC;
```

Explanation. `data_length + index_length` in bytes, converted to MB. The single most useful metadata query for storage planning.

Storage engine per table

Scenario. You want to confirm every table is on InnoDB (for transactions).

Question. How do you check storage engine per table?

```
SELECT table_name, engine
FROM information_schema.tables
WHERE table_schema = 'company_db';
```

Explanation. The `engine` column reveals InnoDB, MyISAM, MEMORY, etc. Anything not InnoDB is worth investigating for transaction safety.

WARNING

> **Expert:** INFORMATION_SCHEMA statistics like `table_rows` and `data_length` are estimates for InnoDB and can be stale by up to `information_schema_stats_expiry` seconds (default 24 hours in MySQL 8). Run `ANALYZE TABLE t;` to refresh the estimates when you need accurate reads.

COMMON MISTAKE

> **Expert:** Comparing `table_schema` against `DATABASE()` inside a stored procedure without quoting or NULL-safety. If no default database is set, `DATABASE()` returns NULL and `table_schema = NULL` matches nothing. Always specify the schema explicitly in metadata queries.

MEMORY TRICK CARD

> **Expert:** `INFORMATION_SCHEMA` is the database's **self-portrait**. Every question of the form 'what does my schema look like?' is answered by one `SELECT` into this namespace.

Figure 6.2 — INFORMATION_SCHEMA as the master catalog — A hub-and-spoke diagram on a light background. In the center a large gold circle labeled 'INFORMATION_SCHEMA' with a small book icon inside. Nine spokes radiate outward to smaller labeled circles: `schemata`, `tables`, `columns`, `statistics`, `key_column_usage`, `table_constraints`, `views`, `routines`, `triggers`. Each spoke has a tiny SQL snippet next to it in a subtle gray font: e.g. `SELECT * FROM columns WHERE table_schema='company_db'` next to `columns`; `SELECT ... FROM key_column_usage WHERE referenced_table_name IS NOT NULL` next to `key_column_usage`. A caption below reads 'One `SELECT` statement can answer any question about your schema — no more grepping through migration scripts.'

MODULE 16 - PERFORMANCE

Every senior developer has the same reflex: before optimizing a slow query, prefix it with `EXPLAIN`. This module walks you through reading execution plans, spotting full table scans, forcing index usage, and rewriting queries the way MySQL prefers. This is the module that separates a Junior from a Senior at ABC Technologies.

EXPLANATION

> **Expert:** MySQL's optimizer plans every query before executing it. `EXPLAIN` shows you that plan — which tables it reads, in what order, which indexes it uses, and roughly how many rows it expects to examine. Reading an `EXPLAIN` plan is a learned skill: once you can do it, you diagnose 90% of slow queries in seconds.

CONCEPT CARD

> **Expert:** Key `EXPLAIN` columns:

- **id** — sequence in complex queries.
- **select_type** — SIMPLE, PRIMARY, SUBQUERY, DERIVED, UNION.
- **table** — the table being accessed.
- **type** — the access method. From best to worst: system, const, eq_ref, ref, range, index, ALL. **ALL means full table scan — usually a red flag.**
- **possible_keys / key** — which indexes are candidates, and which was chosen.
- **rows** — estimated rows to examine.
- **Extra** — flags like 'Using where', 'Using index', 'Using filesort'.

EXPLAIN a slow query

Scenario. The employee-search page is slow. You want to know why.

Question. How do you inspect the execution plan of a query?

```
EXPLAIN
SELECT * FROM employees
WHERE email = 'ravi@company.com';
```

Explanation. Prefix any SELECT with EXPLAIN to see the plan. If **type** is **ALL** and **key** is NULL, MySQL is doing a full table scan. Add an index on **email** and EXPLAIN again — you should see **type = ref**.

Detect a full table scan

Scenario. A dashboard query hits the orders table and feels slow.

Question. How do you tell if a query is doing a full table scan?

```
EXPLAIN
SELECT * FROM orders
WHERE amount > 1000;
```

Explanation. Look at the **type** column. **ALL** means full table scan. If the WHERE clause selects a small fraction of rows, an index on **amount** will drop **type** to **range** and make the query fast.

Verify an index is being used

Scenario. You created an index on **city** but you want to confirm the query is using it.

Question. How do you check if a specific index is picked?

```
EXPLAIN
SELECT * FROM employees WHERE city = 'Hyderabad';
```

Explanation. `possible_keys` lists indexes MySQL considered; `key` shows the one it chose. If your index appears in `key`, it is being used.

Optimize WHERE by matching an index

Scenario. A slow query uses `WHERE YEAR(hire_date) = 2024`. Rewrite it to use an index on `hire_date`.

Question. How do you rewrite a function-wrapped WHERE to be index-friendly?

```
-- slow: function wraps the column
SELECT * FROM employees WHERE YEAR(hire_date) = 2024;

-- fast: same result but index-friendly
SELECT * FROM employees
WHERE hire_date >= '2024-01-01'
  AND hire_date < '2025-01-01';
```

Explanation. Wrapping a column in a function (`YEAR(hire_date)`) blocks index usage. Rewriting as a range makes the index usable and drops the plan to `range`.

Optimize ORDER BY with an index

Scenario. `ORDER BY salary DESC` shows 'Using filesort' in EXPLAIN Extra. Filesort is slow.

Question. How do you avoid filesort on ORDER BY?

```
CREATE INDEX idx_employees_salary_desc
ON employees(salary DESC);

-- now filesort is gone
SELECT first_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 10;
```

Explanation. An index that matches the ORDER BY direction lets MySQL walk the index instead of sorting. The Extra column drops 'Using filesort' and the query becomes near-instant.

Optimize GROUP BY with a composite index

Scenario. `SELECT department_id, COUNT(*) FROM employees GROUP BY department_id` is slow on a large table.

Question. How do you speed up a GROUP BY?

```
CREATE INDEX idx_employees_dept
ON employees(department_id);

SELECT department_id, COUNT(*)
FROM employees
GROUP BY department_id;
```

Explanation. An index on the GROUP BY column lets MySQL group rows without a temporary table. EXPLAIN should show 'Using index for group-by' — the fast path.

Avoid SELECT * for large tables

Scenario. The reporting query pulls every column of employees but only needs three. Trim it.

Question. Why is SELECT * a performance concern?

```
-- wasteful
SELECT * FROM employees WHERE city = 'Hyderabad';

-- lean and covering-index friendly
SELECT first_name, last_name, salary
FROM employees
WHERE city = 'Hyderabad';
```

Explanation. Explicit columns reduce network transfer and enable **covering indexes** — an index containing every column the query needs, letting MySQL skip the table entirely.

EXISTS vs IN for subqueries

Scenario. A query uses `WHERE customer_id IN (SELECT customer_id FROM orders)` and is slow.

Question. When should you prefer EXISTS over IN?

```

-- often slower with large subqueries
SELECT customer_name
FROM customers c
WHERE c.customer_id IN (SELECT o.customer_id FROM orders o);

-- often faster; short-circuits on first match
SELECT customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
);

```

Explanation. EXISTS stops scanning as soon as one match is found. On large subquery outputs it beats IN. Modern MySQL optimizers narrow the gap, but EXISTS is still often faster.

JOIN vs Subquery

Scenario. A junior wrote a correlated subquery. You suspect a JOIN would be faster.

Question. How do you rewrite a correlated subquery as a JOIN?

```

-- correlated subquery: one execution per outer row
SELECT c.customer_name,
    (SELECT SUM(o.amount) FROM orders o
     WHERE o.customer_id = c.customer_id) AS spend
FROM customers c;

-- JOIN + GROUP BY: single pass
SELECT c.customer_name, SUM(o.amount) AS spend
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name;

```

Explanation. A correlated subquery runs the inner SELECT once per outer row. Rewriting as JOIN + GROUP BY collapses everything into one pass — often orders of magnitude faster.

Pagination optimization with keyset

Scenario. `LIMIT 1000000, 10` is painfully slow because MySQL still scans the first million rows.

Question. How do you paginate large tables efficiently?

```
-- slow at deep pages
SELECT * FROM orders ORDER BY order_id LIMIT 1000000, 10;

-- fast keyset pagination
SELECT * FROM orders
WHERE order_id > 1000000 -- last order_id from previous page
ORDER BY order_id
LIMIT 10;
```

Explanation. Keyset pagination uses the last-seen key from the previous page instead of OFFSET. The index jumps straight to the position — constant time regardless of how deep the page is.

Analyze table statistics

Scenario. The optimizer keeps making poor plan choices — statistics may be stale.

Question. How do you refresh MySQL's index statistics?

```
ANALYZE TABLE employees, orders, customers;
```

Explanation. `ANALYZE TABLE` recomputes index cardinality statistics. The optimizer then makes better decisions. Run after large data changes or when EXPLAIN plans look wrong.

Use EXPLAIN ANALYZE for actual timings

Scenario. You want to see not just the plan but the actual measured time per step.

Question. How do you get actual runtime measurements per step?

```
EXPLAIN ANALYZE
SELECT c.customer_name, SUM(o.amount) AS spend
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name;
```

Explanation. `EXPLAIN ANALYZE` (MySQL 8.0.18+) actually runs the query and reports measured rows and elapsed time per step — the gold standard for tuning.

WARNING

> **Expert:** Never optimize without measuring. Add an index only after EXPLAIN shows a full scan on the WHERE column AND the query is actually slow. 'Optimization' that adds unused indexes just slows every INSERT and wastes disk. Rule: measure, change one thing, measure again.

COMMON MISTAKE

> **Expert:** Reading EXPLAIN and celebrating `type = index`. That means MySQL is scanning the **whole index**, not doing a targeted lookup. `type = ref` or `range` is what you actually want. `index` is better than `ALL` but still not the targeted access pattern.

MEMORY TRICK CARD

> **Expert:** Performance debugging checklist — say it out loud on every slow query: **P.I.E.:** Profile with EXPLAIN, Index the WHERE/JOIN/ORDER BY columns, Eliminate SELECT * and function-wrapped predicates. If P.I.E. does not fix it, then rewrite the query shape (JOIN vs subquery, keyset pagination).

QUIZ

> **Expert: Q:** An EXPLAIN plan shows `type = ALL`, `key = NULL`, `rows = 1,200,000`. What is your first action?

A) Rewrite the query with EXISTS instead of IN. B) Add an index on the WHERE column and re-run EXPLAIN. C) Increase the buffer pool size.

Answer: B. `type = ALL` + `key = NULL` is the textbook signal for a missing index. Fix the index first, then re-measure.

EXAM TIP CARD

> **Expert:** Interview classic: 'Walk me through how you would optimize a slow query.'
Answer: 1) EXPLAIN, 2) look at `type` and `key`, 3) if `ALL`, add an index; if `filesort`, index the ORDER BY; if Using `temporary`, index the GROUP BY. Then consider query rewrite — SELECT specific columns, replace correlated subqueries with JOINS, use keyset pagination.

EXERCISE

- > **Expert:** 1. EXPLAIN the slowest report you can find and identify one thing to improve.
2. Take a correlated subquery from Module 8 and rewrite it as a JOIN.
3. Convert a `LIMIT 100000, 10` pagination query to keyset style.
4. On a test copy of employees, create an index on hire_date and use EXPLAIN to confirm the range-scan improvement.

AI PROMPT

> **Expert:** Try this in Claude: 'Given a MySQL query `SELECT c.customer_name FROM customers c WHERE c.city = \'Hyderabad\' AND c.customer_id IN (SELECT o.customer_id FROM orders o WHERE o.amount > 5000)` on a table with 10 million customers and 100 million orders, propose two indexes and a JOIN-based rewrite, then explain why each helps.' Verify the answer references composite indexes and mentions EXISTS or JOIN.

Figure 6.3 — EXPLAIN access types from fastest to slowest — A vertical ladder diagram on a light background. Top rung labeled 'system - one row (system table)' in bright green. Next 'const - matched by PRIMARY or UNIQUE, one row' in green. Next 'eq_ref - one row per outer row, via unique index' in green. Next 'ref - non-unique index lookup, few rows' in light green. Next 'range - index range scan (BETWEEN, <, >)' in yellow. Next 'index - full index scan (still faster than table scan)' in orange. Bottom rung labeled 'ALL - FULL TABLE SCAN' in red with a warning icon. Down the right side a large arrow labeled 'Slower' points from top to bottom. Caption below reads 'Aim for ref or better on hot queries. See ALL? Add an index.'

CHAPTER REVIEW & SUMMARY

EXPLANATION

> **Expert:** Section 6 closed the loop from query author to database engineer. Indexes turn linear scans into logarithmic lookups — the single biggest win in any query. Metadata via INFORMATION_SCHEMA gave you the self-portrait of the database you can query programmatically. And performance tuning with EXPLAIN taught you to diagnose slow queries the way seniors do: read the plan, spot the ALL, add the right index, rewrite the shape. Together, these three modules give you the toolkit to keep ABC Technologies' reports fast even as the data grows.

CONCEPT CARD

> **Expert:** Key takeaways:

- Indexes on WHERE, JOIN, and ORDER BY columns; not everywhere.
- Composite index order: equality columns first, then range/sort.
- INFORMATION_SCHEMA answers every 'what exists in my schema' question.
- EXPLAIN before optimizing; `type = ALL` is the classic red flag.
- SELECT specific columns to enable covering indexes.
- Prefer JOIN over correlated subquery when possible.
- Keyset pagination beats OFFSET on large tables.

You have now completed the SQL journey — from your first `SHOW DATABASES` to reading execution plans like a senior. Welcome to the ABC Technologies engineering team.

Conclusion

You joined ABC Technologies six sections ago knowing nothing about databases. You leave with the toolkit of a working software engineer. You know how to explore an unfamiliar schema in under a minute using SHOW commands and INFORMATION_SCHEMA. You can pull any subset of rows with SELECT, WHERE, DISTINCT, LIMIT, and ORDER BY. You aggregate with COUNT, SUM, AVG, MIN, MAX and slice those aggregates by any column with GROUP BY and HAVING. You combine multiple tables with INNER, LEFT, SELF, and CROSS joins and know when each is appropriate. You handle every 'above average', 'Nth highest', and 'per-row personalized' question with subqueries. You produce rankings, running totals, and per-group top-N results with window functions.

Beyond queries, you build. Views hide complexity behind friendly names. Functions encapsulate business formulas. Procedures package multi-step tasks with clean signatures. Transactions guarantee that critical multi-statement operations either all succeed or all fail. Indexes turn linear scans into logarithmic lookups. And EXPLAIN plans let you diagnose slow queries the way senior engineers do — read the plan, spot the full table scan, add the right index, rewrite the shape.

The next step is practice. Take an open dataset — sample sales data, a public API dump, your own tracker — load it into MySQL, and rebuild the same six sections' worth of queries on your own data. The muscle memory you build there is what will make you productive on your first day at ABC Technologies or anywhere else.

Welcome to the profession. The rest of your career is just larger versions of the questions you have already answered here.